IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 17 March 2025, accepted 11 April 2025, date of publication 17 April 2025, date of current version 30 April 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3562125

SURVEY

Observability in Microservices: An In-Depth Exploration of Frameworks, Challenges, and Deployment Paradigms

UMMAY FASEEHA[1,2], (Member, IEEE), HASSAN JAMIL SYED[3], FAHAD SAMAD[4], SEHAR ZEHRA[1], AND HAMZA AHMED[1]

[1]Department of Computer Science, National University of Computer and Emerging Sciences (FAST), Karachi 75030, Pakistan
[2]Department of Computer Science and Software Engineering, Jinnah University for Women, Karachi 74600, Pakistan
[3]Asia Pacific University of Technology and Innovation (APU), Kuala Lumpur 57000, Malaysia
[4]Department of Cyber Security, National University of Computer and Emerging Sciences (FAST), Karachi 75030, Pakistan

Corresponding author: Hassan Jamil Syed (hassan.jamil@apu.edu.my)

This work was supported by Asia Pacific University of Technology and Innovation (APU), Kuala Lumpur, Malaysia.

ABSTRACT The widespread adoption of microservices-based architectures in native cloud systems has amplified the need for robust observability strategies to ensure system reliability and performance. Microservices, while enabling scalability and flexibility, introduce complexities such as distributed failure points, performance bottlenecks, and resource mismanagement. This paper provides a comprehensive review of state-of-the-art observability frameworks and tools designed for microservices architecture. It focuses on key aspects of observability at the system, service, and network levels, utilizing logs, traces, and metrics as core data sources. A thematic taxonomy is proposed, classifying the diverse approaches to monitoring microservices based on implementation paradigms, deployment platforms, and architecture patterns. The study compares existing solutions in terms of their capabilities, limitations, and effectiveness in addressing observability challenges in cloud, edge, and fog environments. Furthermore, open research issues are identified, and practical recommendations are provided to optimize observability in complex, distributed microservices ecosystems. This work aims to serve as a valuable resource for system architects, DevOps engineers, and researchers striving to enhance the reliability and performance of modern cloud-native systems.

INDEX TERMS Microservices observability, cloud-based microservices, system reliability, containerized computing environments, real-time performance monitoring.

I. INTRODUCTION

The rapid shift of IT infrastructure towards distributed, complex, and dynamic systems, driven by the increasing number of data sources, highlights the growing need for effective observability. Managing resources in such decentralized systems requires advanced tools to address challenges like distributed failures and resource inefficiencies [1]. The integration of edge and cloud computing plays a key role in improving observability by offering real-time monitoring, better system visibility, and more efficient management. These advancements help ensure the reliability and performance of microservices-based cloud-native systems in increasingly complex environments [2], [3]. Microservices are a widely used distributed platform [4], [5], with well-known use cases of companies shifting from monolithic to microservices architecture including Amazon, Netflix, Uber, and Etsy [6], [7], [8]. Containers are the cornerstone of microservices architecture by encasing applications and their dependencies into flexible and portable modules. They enable isolation, ensuring each microservice operates independently while facilitating portability across diverse environments. Containers streamline scalability, resource efficiency, and version management, supporting continuous deployment and integration pipelines. With fault isolation and support for DevOps practices, containers empower

The associate editor coordinating the review of this manuscript and approving it for publication was Renato Ferrero.

organizations to build, deploy, and scale microservices effectively, driving agility and innovation in modern cloud-native architectures. Microservices architectures consist of multiple small, independently deployable services housed in containers, each assigned a specific task [9]. The number of microservices can vary depending on the application's complexity and requirements. These small services are easily deployable and highly multilingual, offering excellent reliability and performance in distributed and scalable infrastructures [10]. With the increasing adoption of microservices in the current environment, computational complexities will rise, potentially leading to system performance degradation, service failures, and mismanagement in allocating desired resources [11], [12]. Real-time active observability and management of microservices architectures enhance the system's reliability and performance [13], [14]. Observability is based on three main elements as illustrated in fig 1 logs, traces, and metrics [15], [16]. Logs are the simple text-based files containing records of system events and errors [17], [18], which is a fundamental part of the observability of the complex distributed architecture. However, it generates a massive amount of data. Traces are integrated within the complex system to identify bottlenecks, end-to-end visibility of the request, and faulty application components without necessarily identifying the cause of the failure [10], [19]. These metrics help gauge problem severity, provide visibility, and trigger alerts in case of unusual system performance [20], [21]. With the growing prevalence of microservices in modern applications, effectively understanding and monitoring the diverse range of logs they produce can be challenging [22], [23]. For troubleshooting in case of any failure, establishing a trace between these logs is crucial [24]. In such systems, diverse levels of observability are necessary to gain deeper insights; among these layers, three primary levels of observability have been the focus of most research efforts: system level, service level, and network level, each emphasizing a distinct element of health and performance [25], [26], [27].

System level observability ensures the actual application health and the performance of the system, including communication of the components using API gateway, load balancers, and network traffic information, which depend upon the collected data, for instance, logs, metrics, and traces [28]. This level of observability is needed to evaluate the system's resilience, scalability, and responsiveness for smooth communication and interaction between services. Conversely, Service level observability focuses on monitoring individual microservices, which includes their performance, health, and behavior aspects such as response times, error rates, and resource usage. The primary goal is to ensure that each microservice operates appropriately and efficiently, meeting its specified service level objectives (SLOs). This is crucial for identifying issues or fine-tuning system components [29]. In microservices-based applications, network-level observability includes monitoring and

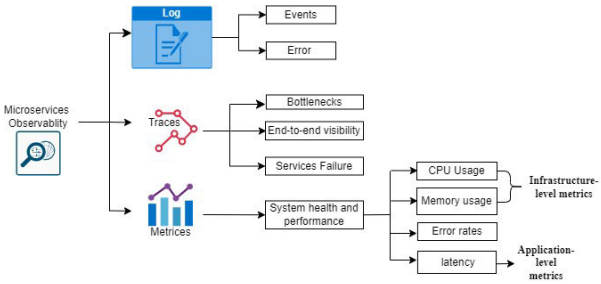

FIGURE 1. Observability components in microservices architectures.

analyzing the network connections' communication patterns, performance metrics, and security aspects between individual microservices [30]. This observability better detects communication issues such as network bottlenecks, high latency, and congestion, facilitating timely troubleshooting and remediation. Monitoring network traffic and metrics can identify performance anomalies, and resources can be optimized to ensure scalability and reliability [31]. Additionally, network-level observability is critical in enhancing security by identifying and alleviating possible threats. Integration with logging and monitoring systems offers a comprehensive view of the application's behavior, allowing effective troubleshooting and root cause analysis.

Edge computing introduces unique observability challenges due to its distributed nature, limited resources, and low-latency requirements. Unlike cloud computing, which benefits from centralized resources and mature monitoring tools, edge computing disperses resources across nodes, including edge devices, fog nodes, and centralized cloud servers [32]. This distributed architecture necessitates specialized observability solutions to maintain system reliability and efficiency while optimizing resource-constrained edge devices [33], [34], [35], [36], [37].

Service-level observability takes precedence in edge environments due to the frequent workload shifts and real-time processing demands [38], [39]. Services deployed on edge nodes often migrate between nodes to adapt to changing workloads, emphasizing the need for granular performance monitoring to ensure minimal latency [40]. Real-time observability tools must account for these dynamic behaviors to support critical applications such as IoT devices and edge analytics [41].

Fog computing, as an intermediary layer between edge and cloud, adds further complexity by requiring observability across heterogeneous environments with diverse workloads. Monitoring service performance across fog nodes introduces challenges in managing resource utilization and identifying bottlenecks [42]. Specialized tools are needed to address these complexities, offering insights into communication patterns, performance metrics, and potential security threats [43]. These tools must integrate seamlessly across the edge-fog-cloud continuum to provide a unified observability framework.

With minimal performance overhead, eBPF emerges as an ideal observability tool for distributed systems, allowing real-time insight between edge devices, fog nodes, and cloud infrastructure [44], [45], [46]. Its ability to operate directly within the Linux kernel makes it particularly well-suited for heterogeneous environments. It enables the execution of one's applications at various places in the kernel, such as network sockets or system calls, without requiring changes to the kernel source code. This allows the behavior of the kernel to be tailored dynamically and flexibly and is therefore considered one of the significant improvements [47] in the way, the kernel works with user-defined code. The tools being used today for the monitoring and visualization of different aspects from a user-space level, for example, Grafana, Prometheus, BCC, and others [48], use different methods to gather and analyze data from the various layers that comprise the software stack, such as applications, services, and operating system metrics. In contrast, eBPF runs within the runtime of the Linux kernel right out of the box, enabling the execution of custom programs inside the kernel. This allows for tracing, monitoring, and interaction with events and activities occurring in the kernel [49].

In this paper, we comprehensively surveyed state-of-the-art solutions for microservices observability. Our survey stands out by presenting a detailed taxonomy that categorizes various aspects of observability, including monitoring parameters, implementation paradigms, deployment platforms, tools, and architecture patterns. This provides a robust comparative analysis of state-of-the-art solutions, offering both qualitative and quantitative evaluations, which provide a nuanced understanding of various observability tools. Unlike various monitoring frameworks based on domains other than microservices, such as in the article [50] the researchers Muvskardin et al. focused on observing the agent's performance in terms of its decision making and adaptability in partially observable environments, Min et al [51] offered smart grid observability using stochastic machine learning-based approach. In addition, Wu et al. [52], have analyzed the structure and communication patterns of the IoT network between nodes (vertices) to identify key vulnerabilities, in the article [53], the authors have analyzed the observability of traffic flow and sensor networks within the context of the 6G enabled Internet of Vehicles (IoV). While [54] Cong et al. addressed critical observability in discrete event systems using labeled Petri nets, commonly used in manufacturing, telecommunications, traffic and computer networks, and Akbari et al. [55] monitored and analyzed resource utilization and energy efficiency in mobile networks 5G and Beyond 5G (B5G), which focus on other areas than microservices such as IoT security, machine learning and deep learning environment, 6G (IoV), or sustainability in B5G networks. In addition, we identify open research challenges and issues that effectively guide future research directions. The main contributions of this paper are as follows: i) It surveys state-of-the-art observability solutions for microservices, providing detailed technical and practical insights into these frameworks. ii) It proposes a thematic taxonomy to classify the existing literature into distinct categories. iii) The research methodology section clarifies the main objective of the research and paper selection method. iv) It analyzes current solutions based on this proposed taxonomy. v) Comparative Analysis of State-of-the-Art Microservices Observability Solutions, vi) Evaluation of Microservices Observability Solutions: Tools, Features, and Performance Metrics, and vii) Lastly, it highlights open research issues and challenges in microservices observability, offering future guidance for researchers in the field.

II. RELATED WORK

This section highlights surveys related to various aspects of microservices, discussing the contributions, limitations, and motivation of the researchers behind this study.

A. CONTRIBUTIONS AND LIMITATIONS OF THE EXISTING LITERATURE

An overview of existing research reveals significant contributions and notable gaps in the field of microservice observability. Usman et al. [56] conducted a survey on observability solutions, but their study was confined to edge environments, addressing the challenges of distributed edge infrastructures. Kosińska et al. [27] conducted a systematic mapping of cloud-native observability based on studies from 2018 to 2022, providing valuable insights but potentially missing the latest trends. Li et al. [16] presented an industrial survey on microservice tracing, emphasizing challenges faced by developers, though its narrow focus on specific practices limit generalizability. Parmar et al. [57] analyzed observability design patterns and tools, focusing on adaptability to project needs, while Li et al. [58] reviewed quality attributes of microservice architectures, identifying critical factors but overlooking emerging trends. Velepucha and Flores [59] compared microservices' principles and patterns with monolithic architectures, offering a theoretical perspective. Lastly, Zhang et al. [60] provided an extensive survey on microservice system failure diagnosis, discussing key frameworks and concepts but lacking a focus on recent practical developments. Collectively, this literature provides valuable insights but reveals limitations in scope and depth, highlighting the need for a more comprehensive analysis, as undertaken in this research study. Collectively, the studies summarized in table 1 provide valuable insights but reveal limitations in scope and depth, highlighting the need for a more comprehensive analysis, as undertaken in this research study.

B. RESEARCH MOTIVATION

The motivation for our research comes from the growing complexity and importance of observability in modern microservices architecture [61], [62], [63]. In today's interconnected systems, where many small services operate independently, understanding how they interact is crucial.

TABLE 1. Summary of observability domains, contributions, and shortcomings of each survey paper.

Ref.	Observable Domain	Contributions	Shortcomings
[56]	Edge Computing and Distributed Microservices	Surveyed state-of-the-art solutions; discussed future directions.	Limited to edge environments; lacks actionable future directions.
[27]	Cloud-Native Applications	Maps Cloud-native observability; assessed motivations and trends.	May not include latest advancements; limited to 2018-2022 studies.
[16]	Microservice Tracing and Analysis	Highlighted practical challenges and gaps in tracing techniques.	Narrow focus; limited on advanced methods.
[57]	Observability Design Patterns	Discussed design patterns and tools; emphasizes adaptability.	Generalized; lacks specific tool details.
[58]	Quality Attributes of Microservices	Reviewed six key Quality Attributes (QAs) and related tactics.	May overlook emerging QAs; limited on practical implications.
[59]	Microservices vs. Monolithic Architectures	Compares microservices with monolithic; surveyed design patterns.	Theoretical; lacks practical case studies.
[60]	Failure Diagnosis in Microservices	Reviewed failure diagnosis techniques; synthesized various papers.	May not cover recent developments; lacks practical insights.

Observability tools allow us to gain deep insights into these interactions, which is essential for maintaining system reliability and performance. As technology evolves, the role of observability in preventing system failures and ensuring smooth operation becomes even more critical. This inspired us to explore how observability frameworks can contribute to the efficiency and stability of microservices.

These challenges make our contribution on microservices' observability unique with its holistic taxonomy: we classify all of the concerns related to observability—monitoring parameters, implementation paradigms, deployment platforms, tools, and architecture patterns—in a highly detailed manner. Our study provides a qualitative and quantitative comparison of state-of-the-art solutions for putting different observability tools into perspective with nuanced understanding. It also focuses on very recent advancements, identifies open research issues, and challenges future research directions. The completeness of this practically explorative survey of the observability frameworks make the survey somewhat different from other studies that may give more generalized insights or have quite limited temporal scopes, further underlining the importance of addressing these emerging complexities.

III. RESEARCH METHODOLOGY

To effectively analyze the state of observability frameworks and tools in microservices architectures deployed on containerized environments, a structured methodology was essential. This approach considers the added complexity introduced by these systems and aims to generate new insights into their mechanisms concerning observability. The methodology highlights the development of a thematic taxonomy and comparative analysis as key contributions, ensuring the findings offer actionable scientific value.

A. RESEARCH OBJECTIVE

This study provides a comprehensive review of observability frameworks and tools tailored for containerized environments targeting microservice architecture. A primary contribution of this work is the development of a thematic taxonomy to systematically classify existing frameworks and tools. This taxonomy is designed to address the lack of structured classifications in the field, enabling a clear understanding of the functionalities and applicability of each framework. Furthermore, the study offers a comparative analysis of these frameworks, providing insights into their roles in improving system reliability and performance. By delivering actionable insights, this work aims to contribute to the advancement of observability practices within increasingly complex and dynamic microservices environments.

B. DATA COLLECTION PROCESS

Searches were conducted using IEEE Xplore, ACM Digital Library, and Google Scholar, utilizing targeted keywords such as:

- Observability in microservices
- Containerized environments
- Microservices Performance Monitoring in Kubernetes
- Observability in Cloud-Native Applications

Additionally, industry-standard tools such as OpenTelemetry, Prometheus, and Grafana were selected due to their widespread adoption. The collected data informed the thematic taxonomy and facilitated the comparative analysis of observability solutions.

C. SELECTION OF OBSERVABILITY FRAMEWORKS AND TOOLS

The selection process prioritized frameworks proposed by researchers and the tools used within these frameworks, ensuring relevance to microservices and containerized environments. Industry-standard solutions such as Prometheus, Loki, and Jaeger were included, along with tools designed for containerized computing environments to address emerging challenges. The evaluation focused on three key observability features:

- Metrics collection
- Distributed tracing
- Log aggregations

These criteria provided comprehensive insights into each framework's role in enabling observability within microservices architectures.

D. INCLUSION AND EXCLUSION CRITERIA

The following criteria guided the selection of studies and frameworks:

Inclusion Criteria:

- Studies focusing on observability frameworks for microservices in containerized environments.
- Research addressing tools and techniques for metrics collection, distributed tracing, and log aggregation.
- Papers presenting quantitative or comparative analyses of observability solutions.
- Frameworks proposed or designed by the researcher for containerized computing environments.

Exclusion Criteria:

- Studies focusing exclusively on traditional monolithic architectures.
- Research limited to container orchestration without addressing observability.
- Outdated studies (published before 2019) unless they introduce foundational concepts.
- Solutions that do not support cloud-native architectures or distributed environments.

E. TAXONOMY DEVELOPMENT

The thematic taxonomy was created by grouping the tools and frameworks observed based on their key features. It considered factors such as the purpose of monitoring (e.g., performance analysis, issue detection, root cause analysis), monitoring parameters (e.g., logs, metrics, traces), and monitoring levels (e.g., network, system, or service level). The taxonomy also included various deployment platforms like cloud, fog, and edge, as well as architectural patterns (e.g., service mesh, event-driven) and monitoring tools (e.g., Prometheus, Grafana, OpenTelemetry). This process involved a thorough analysis and comparison of different tools and frameworks to ensure a comprehensive and coherent classification system.

IV. TAXONOMY OF MICROSERVICES OBSERVABILITY

In this section, we introduce a thematic taxonomy that organizes existing solutions for observability in microservices by drawing on common parameters identified in the literature. A visual classification of these solutions is presented in fig 2, highlighting several key factors essential for understanding their diverse applications. This classification includes: (i) The purpose of Monitoring, which defines the main objectives such as performance assessment or fault detection; (ii) Monitoring Parameters, focusing on the specific metrics and data collected; (iii) Monitoring Scope, which differentiates between various operational layers like infrastructure and application; (iv) Implementation Layers, detailing the approaches used in deploying monitoring solutions; (v) Deployment Environment, identifying the environments like Kubernetes or Docker; (vi) Monitoring Tools, which lists the specific tools and technologies employed; and (vii) Architecture Pattern, describing the structural design of the monitoring solutions. This comprehensive framework, depicted in fig 2, aids in systematically analyzing and comparing different observability solutions within the microservices ecosystem.

A. PURPOSE OF MONITORING

In the case of microservices observability, monitoring is an essential approach for making distributed systems robust and powerful. Monitoring shall include anything from performance to detection and diagnosis of any failure, anomaly, and overall health assessment of applications. This proactive way helps an organization identify issues, solve them in the shortest time, optimize resources, and enhance resilience and flexibility in microservices architecture. Each of these categories is explored individually.

1) PERFORMANCE ANALYSIS

Performance analysis deals with the continuous monitoring of the different metrics, which may include response times, throughput, and service availability. This is generally done to understand how microservices process requests that come their way and the work processing management. Response time monitoring gives an insight into how much time a service takes to process requests and provide results. Faster response times normally mean that things are working well and users are having a superior experience. By monitoring these times, an organization can observe which services are performing slowly and thus need optimization. Throughput defines the amount of work that a service completes within a given period, such as the number of requests processed every second. This metric is important in terms of judging microservices on their capacity and efficiency. High throughput means that a service can process many requests without hindrance, while low throughputs might echo bottlenecks or resource limitations. Furthermore, by monitoring service availability, one is assured that services will be available and accessible at the time needed. High availability is significantly important to retain users' trust and satisfaction with service providers. The tracking of downtimes and halting of services should provoke any organization into being more reliable and avoiding such scenarios. Performance analysis will let the organization understand the performance trends, detect anomalies, and understand how changes affect the performance of services. This enables ongoing improvement and optimization of microservices.

2) FAILURE ROOT CAUSE ANALYSIS

Root cause failure analysis involves the in-depth investigation of errors, exceptions, and events leading to a particular failure. Its main aim is to establish the root cause of a problem so that it does not recur again in the future. An organization can seek to know where and why a certain failure happened

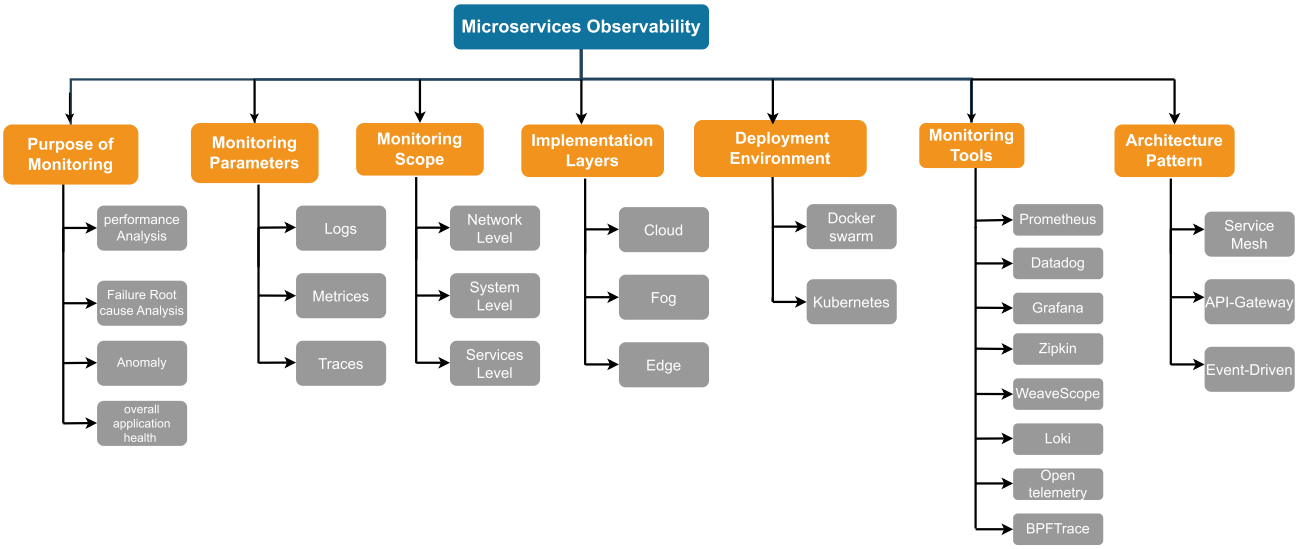

FIGURE 2. Thematic taxonomy of microservices observability.

by gathering logs recording the occurrences of errors and exceptions. These logs provide wonderful information on the nature of the error, the components involved, and the context under which the issue occurred. Understanding the event sequence leading to failure is critical in locating the root cause. Event correlation allows for the tracing of interactions and dependence among microservices so one can see how a problem in one part of the system might have caused failures somewhere else. To that effect, there is usually a technique known as distributed tracing, which helps with the visualization of flows of requests between microservices. The evaluation of impacts brought about by such failures presents priorities of fixes in terms of severity and extent of users impacted. By identifying what services or users are impacted, an organization is better set to focus resources on resolving the most critical issues first. The ultimate goal of a root cause analysis shall be recommending prevention for a similar focus of failures likely to occur in time to come. The kind of solution provided may be a bug fix, improvement in error handling, or enhancement of general robustness on microservices. Organized handling of the root cause can help an organization increase the reliability and stability of its systems.

3) ANOMALY DETECTION

Anomaly detection is the process of finding patterns that are atypical of normal/usual system behavior, hence pointing to problems in the system. Such problems include security breaches or malfunctioning of the systems. Approaches towards anomaly detection include machine learning, statistical methods, and threshold-based alerting systems. Statistical methods of anomaly detection work in ways that include looking at past data to define what normal looks or feels like. By knowing these trending patterns, the system can flag those anomalies that fall out of that trend. For example, if a server's CPU normally ranges between 20-40%, sudden use of 90% could be identified as an anomaly. Common statistical techniques include the calculation of z-scores, which measure the number of standard deviations an observation is away from the mean, and moving averages, smoothing out short-term fluctuations to show longer-term trends.

The detection of anomalies with machine learning algorithms is much finer. This can be achieved by learning from the data over time and hence increasing the accuracy of the anomalies identified. It often makes use of various techniques such as clustering, classification, and neural networks. Clustering algorithms, for example, k-means, cluster similar data points and identify outliers that do not fit into any cluster. Anomaly detection is carried out by training classification algorithms such as decision trees or support vector machines on labeled data to identify typical behavior versus abnormal behavior. Of all, neural networks, especially deep learning models, are quite efficient at handling complex and high-dimensional data, which proves very effective to pick up minor or sophisticated anomalies that may get missed by simpler methods.

Apart from the usual techniques, another method of performing anomaly detection involves threshold-based notifications. This involves setting limits for some metrics, threshold-based approaches, and triggering an alert in the case of a breach of that limit. This approach is straightforward and works well if the system's normal behavior is very stable, but it could be less useful in dynamic environments where normal behavior changes frequently. This may lead to too many false positives when using this method in such scenarios or missed anomalies. Setting a threshold on a network volume, for instance, can quickly flag possible problems, like a DDoS attack.

The advantage of anomaly detection is that it provides early warning of potential entry point issues. By identifying such anomalies in a timely way, organizations can take corrective

steps before issues spiral out of control, thereby minimizing downtime and curtailing risks.

4) OVERALL APPLICATION HEALTH

Overall status analysis of a system begins with health checks and ends with resource utilization, including dependency checks. This will ensure comprehensive optimization of all the microservices within the system. These are periodic tests to ensure the proper functioning of any given microservice. Checks for the status of different components, which include databases, APIs, and third-party services, are conducted. These would include, for example, ping tests that send simple requests to services to see if they are available, and custom health endpoints like health or status that report on the health status of a service. The checks on dependencies check for the health and availability of all those external services and resources the application depends on. Such checks are meant to ascertain that all the dependencies are up, reachable, and performing well. Examples of this include database reachability and responsiveness, as well as the availability and response times of APIs. Resource utilization reviews focus on monitoring CPU, memory, disk space, and network bandwidth usage. This helps identify bottlenecks and potential problems caused by resource shortages. For example, CPU usage is monitored to prevent any service from overloading the processor. Similarly, memory is checked to avoid leaks and ensure efficient usage, disk space is tracked to prevent running out of storage, and network bandwidth is monitored to maintain smooth communication between services. The primary goal of assessing an application's overall health is to ensure that microservices function efficiently. This involves identifying and fixing issues proactively before they affect users, guaranteeing a highly available and performance-driven system. It ensures seamless interaction between services and their dependencies, avoids resource exhaustion, and promotes optimal resource utilization.

B. MONITORING PARAMETERS

The microservices observability parameters: logs, metrics, and traces are essential for understanding the performance, behavior, and health of distributed systems. Each of these elements provides unique insights, helping manage and optimize the microservices architecture effectively.

1) LOGS

Logs include text data generated by microservices, recording various events, the occurrence of error messages, transactions, and changes in system status. The result from log analysis provides a much-valuable understanding of the point where things went wrong, timelines showing what happened to lead to failure, and such request flows among different microservices. The logs provide an ordered record of activities that help to identify the root cause of system events. Therefore, they are useful in detecting anomalies in the system, diagnosing problems, and many other activities critical for ensuring system reliability.

2) METRICS

Metrics provide insights into the performance and resource utilization of microservices. These can be CPU utilization, memory usage, and network throughput or traditional application-level metrics such as active users or several processed transactions per second. Such monitoring of metrics enables the organization to perceive system health, performance trends, and anomalies that may indicate impending problems or optimization opportunities. The metrics are aggregated over time and can be visualized through graphs and dashboards to proactively analyze performance and do capacity planning.

3) TRACES

Traces capture detailed information about every request crossing the system, capturing each involved microservice in turn. Tracing should provide a fine-granular view of request processing, including latency measurements and dependencies between microservices. Only then can an organization analyze traces to identify performance bottlenecks and optimize request paths to improve responsiveness for microservices overall. Especially in large modern complex distributed systems, traces become particularly important when knowledge of the interaction between components provides a means of diagnosing issues and heading toward resolution. Tracing will further enable root cause analysis, performance optimization, and troubleshooting by adding visibility to request flows and service dependencies.

C. MONITORING SCOPE

In the context of observability in microservices, monitoring require at multiple levels to ensure comprehensive insight into the behavior and performance of the system.

1) NETWORK LEVEL

This will monitor the interaction between the involved services, how one microservice sends data to another, and so forth. It also identifies any potential failure points that may lead to breaking this communication chain. You are able to see how much traffic goes between services with the help of the traffic patterns. Latency refers to how long data takes to travel between services. It is another important metric that helps locate delays or bottlenecks. Investigating network issues, such as lost packets or failed connections, can be done by monitoring the rate of communication errors. In addition, network throughput monitoring ensures that data throughput occurs at an acceptable rate, which is critical for the overall performance of the system. That level of monitoring is key to ensuring the network supports seamless operation of microservices.

2) SYSTEM LEVEL

System-level monitoring is focused on the underlying infrastructure whether it is physical servers or virtual machines. The performance here flows directly into the reception

of the microservices. CPU load provides insight into the extent of processing power being utilized, thereby indicating potential resource overloading. Disk I/O monitoring assesses the rate at which data is read from or written to the disk, which is particularly critical for services that rely heavily on stored data for their operations. Monitoring memory usage is essential, as insufficient memory can significantly degrade service performance and, in many cases, lead to complete system failure. Only by closely monitoring these system-level metrics is it possible to keep issues at bay before they start impacting microservices performance, hence maintaining a stable and efficient infrastructure.

3) SERVICES LEVEL

On the services level, monitoring is focused on the application layer, essentially the performance and behavior of each microservice. This form of monitoring is very crucial in handling the general performance of the application. All the services put up will be closely watching error rates so that a bug or failure can be quickly detected very fast. Besides, response times are going to be important to monitor because they show at what speed a service is capable of processing requests and thereby directly influence the user experience. Moreover, transaction volumes need to be tracked to understand the load on each service and its greater ability to deal with rising demand. This is done by looking at the values of these metrics, hence giving important information on the performance of each microservice that leads to proactive management for the smooth and effective running of the whole application. This monitoring level is important in terms of maintaining a reliable, highly performing microservices architecture.

D. IMPLEMENTATION LAYERS

Microservices can be implemented in various environments, each offering unique advantages and catering to specific requirements.

1) CLOUD

The implementation of microservices on cloud infrastructure brings significant advantages, such as scalable resources, high reliability, and extensive geographical coverage. Platforms like AWS, Azure, and Google Cloud offer on-demand resources that adjust to workload fluctuations, enabling applications to handle peak demands efficiently without requiring substantial upfront investments in hardware. Furthermore, the cloud's distributed architecture boosts service reliability by spreading resources across multiple data centers, reducing the risk of downtime. For applications with global audiences or variable demands, the cloud's effective resource management and simplified deployment process make it a highly beneficial choice.

2) FOG

Fog computing extends cloud computing by bringing more computational resources closer to the edge of the network, right near the end user. This decentralized approach helps reduce latency and improve application responsiveness by processing data closer to its origin. In fog computing, intermediate nodes handle tasks like data aggregation and initial processing, which helps reduce bandwidth usage and lessen the load on central servers. This model is particularly effective for real-time applications, such as IoT devices and edge analytics, where quick data processing is essential.

3) EDGE

Edge computing brings computation and data storage closer to the source of the data, like within IoT devices or sensors. Processing tasks locally rather than sending data to central servers, significantly cuts down on latency, improving responsiveness. This method is ideal for applications that require fast data processing and high bandwidth, such as autonomous vehicles and industrial control systems. Additionally, edge computing reduces the need for continuous data transmission to central servers, saving bandwidth and boosting overall efficiency.

In addition to deployment layers (cloud, fog, edge), observability tools can also be categorized based on their implementation paradigms. Open standard tools like Prometheus and OpenTelemetry are known for their flexibility and ability to operate across diverse environments. In contrast, proprietary tools such as Datadog and AWS CloudWatch offer seamless integration but are limited to specific vendors. However, the tools are further discussed under the heading monitoring tools.

E. DEPLOYMENT ENVIRONMENT

Microservices can be deployed using various platforms, each tailored to meet specific operational needs and enhance the management of containerized applications. In this subsection, we examine a set of widely recognized monitoring tools for microservices-based architectures, identified through an extensive review of the literature.

1) DOCKER SWARM

Docker Swarm serves as a native clustering system for Docker, effectively transforming a group of Docker hosts into a singular, cohesive virtual host. Key monitoring aspects should include the status of nodes, containers, and services within the swarm. This platform is particularly advantageous for simplified orchestration, native integration with Docker environments, and streamlined scalability of applications.

2) KUBERNETES

Kubernetes excels in managing and dynamically scaling microservices, using configurations that meticulously define the desired operational state of applications. Monitoring efforts for Kubernetes should focus on assessing pod status, cluster resource allocation, and orchestration behaviors. This platform is renowned for its robustness in handling complex applications, extensive community support, and its capability to ensure continuous availability and resilience.

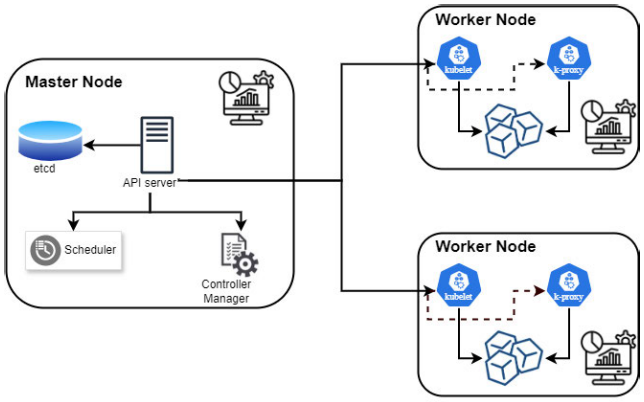

FIGURE 3. Kubernetes cluster.

The architecture of a Kubernetes cluster is shown in fig 3, comprising a master node and various worker nodes. The master node, containing components such as the API server, scheduler, controller manager, and etcd, manages the cluster's overall state and scheduling of workloads. The worker nodes, equipped with kubelet and kube-proxy, are responsible for running the containerized applications and maintaining network communication. The API server acts as the central management interface, while etcd serves as the key-value store for all cluster data. Together, these components ensure efficient and consistent operation of containerized applications within the Kubernetes cluster.

F. MONITORING TOOLS

In this subsection, we examined a set of widely recognized monitoring tools for microservices-based architectures, identified through an extensive review of the literature. These tools were selected based on their frequent implementation by researchers and industry practitioners, as well as their proven effectiveness in enhancing observability, maintaining system health, and ensuring uninterrupted services. They address critical aspects such as logging, tracing, metrics.

1) PROMETHEUS

Prometheus is an open-source monitoring solution developed by SoundCloud in 2012. It's designed for reliability and scalability, primarily handling multi-dimensional data collection and querying. It has a strong ecosystem and integrates with various service discovery options while using a time-series database for storing its data. Prometheus gathers metrics from monitored targets by polling HTTP endpoints on these targets. One of its core features is a powerful query language called PromQL. GitLab and Red Hat are among the major users of Prometheus. They utilize it to monitor their infrastructure and ensure their systems operate efficiently and reliably.

2) DATADOG

Datadog is a SaaS-based monitoring and analytics platform that provides full-stack observability by integrating metrics, traces, and logs. Founded in 2010, Datadog offers extensive automation and monitoring capabilities across cloud services, servers, databases, tools, and services. The tool is known for its real-time performance dashboards and scalable infrastructure monitoring that can handle vast amounts of data and applications. Airbnb and Netflix leverage Datadog for their comprehensive monitoring capabilities that integrate seamlessly with their complex cloud environments.

3) GRAFANA

Grafana is an open-source platform for monitoring and observability, known for its sophisticated visualization capabilities. Launched in 2014, it works seamlessly with a variety of databases, including Prometheus, MySQL, and Elasticsearch, to display data through comprehensive graphs, charts, and alerts. However, To access the advanced features of Grafana Enterprise, an Enterprise license activation is required. Grafana allows users to create customized dashboards according to their monitoring needs. Uber and eBay use Grafana for its powerful visualization capabilities to monitor their vast operations and to help with the analysis of their metrics and logs.

4) ZIPKIN

Zipkin is a distributed tracing system that collects and manages timing data to diagnose latency issues in microservices architectures, facilitating both data collection and retrieval. Originating from Twitter and based on the Google Dapper paper, Zipkin provides crucial insights into application performance and issues related to complex distributed systems. Twitter and major financial institutions like Credit Karma deploy Zipkin to trace transactions through their distributed services and diagnose latency issues effectively.

5) WEAVE SCOPE

Weave Scope is an open-source tool specifically designed for monitoring, visualizing, and managing Docker and Kubernetes containers. It automatically generates a map of your processes, containers, and hosts, allowing you to understand, monitor, and control your applications and containers directly from your browser.

6) LOKI

Loki, created by Grafana Labs, is a horizontally scalable log aggregation system that can expand across multiple servers to manage large data volumes. It ensures high availability, so it remains operational even if some parts fail, and supports multiple tenants, enabling different users or organizations to share the system without overlapping data. Launched in 2018 and inspired by Prometheus, Loki is designed to be cost-effective and user-friendly. Instead of indexing the full content of log entries like traditional systems, Loki indexes logs based on their labels. This label-based indexing reduces storage needs and processing demands, making log management more economical and efficient. This tool

simplifies searching and retrieving logs, minimizing system performance and cost impacts.

7) OpenTelemetry

OpenTelemetry is an observability framework for cloud-native software, providing APIs and libraries to collect distributed traces and metrics from applications. It is a Cloud Native Computing Foundation (CNCF) project that emerged from the merging of OpenCensus and OpenTracing projects. OpenTelemetry aims to make telemetry data (metrics, logs, and traces) a built-in feature of cloud-native software applications.

8) BPFTrace

BPFTrace is a high-level tracing tool for Linux based on BPF, the Berkeley Packet Filter. It was developed as a more accessible interface for dynamic tracing in Linux, providing a powerful and flexible way to observe and modify the behavior of running Linux systems. BPFTrace is especially useful for analyzing performance issues and troubleshooting system calls and events. Facebook and Netflix use BPFTrace for dynamic tracing in their Linux environments to analyze and debug performance issues in real-time.

9) ISTIO

Istio is a comprehensive service mesh that significantly enhances the monitoring of microservices by facilitating detailed insights and control over complex microservices architectures. It automatically collects telemetry data such as metrics, logs, and traces through sidecar proxies, which integrate seamlessly with each service. This enables Istio to provide real-time traffic management, service graphs, and observability without modifying the microservices themselves. Additionally, Istio supports integrations with popular tools like Prometheus, Grafana, and Jaeger, offering advanced capabilities for dashboard visualization, distributed tracing, and anomaly detection. This robust feature set allows for effective monitoring, alerting, and operational visibility, making Istio an invaluable tool for managing distributed microservices environments.

10) JAEGER

Jaeger is an open-source distributed tracing tool that excels in monitoring microservices by providing deep insights into the interactions and performance of complex service architectures. It enables detailed tracking of requests across services, helping to identify bottlenecks, optimize performance, and conduct effective root cause analysis. Jaeger's integration with other monitoring and observability tools like Prometheus, Grafana, and service meshes such as Istio enhances its capability to offer comprehensive visibility without the need for invasive code modifications. With its user-friendly web UI, Jaeger facilitates the visualization and analysis of traces, making it easier for teams to understand service dependencies and data flow, which are critical for maintaining the reliability and efficiency of microservices environments.

G. ARCHITECTURE PATTERN

Architecture patterns play a crucial role in enabling effective observability in microservices. Observability, in the context of microservices, involves the ability to monitor and understand the state and behavior of services, which helps in identifying issues, optimizing performance, and ensuring reliability.

1) API GATEWAY

The API Gateway is a managing layer that acts as the single point of entry for external clients into the microservices ecosystem. This pattern is designed to address the complexity of having multiple endpoints by providing a unified interface to clients. The gateway routes incoming requests to the appropriate microservices while also handling cross-cutting concerns such as authentication, authorization, and rate limiting. Additionally, it can aggregate results from multiple microservices into a single response to simplify the client experience. The use of an API Gateway can facilitate the management of endpoints, especially in large systems with numerous microservices, and it can be pivotal in orchestrating microservices' interactions and optimizing communication flows.

2) EVENT-DRIVEN

In an event-driven architecture, microservices communicate with each other using events. This pattern promotes a high level of decoupling as services do not send requests to each other directly but instead produce and consume events asynchronously. Events are typically emitted as a result of a service performing a task, such as updating a database or processing a user action. Other services listen to these events and react accordingly, which allows for a responsive and flexible system that can easily adapt to changes. This pattern is especially beneficial in dynamic environments where services need to scale independently and remain resilient to the failures of others.

3) SERVICE MESH

A service mesh introduces a transparent layer of infrastructure dedicated to managing service-to-service communication in a microservices architecture. It effectively abstracts the complexity of these interactions by providing a distributed networking layer that has a broad understanding of the architecture and can control communication. This pattern facilitates features like service discovery, load balancing, encryption, observability, and traceability without requiring changes to the individual services. A service mesh employs sidecar proxies that accompany each service, taking responsibility for inter-service communications, which allows developers to focus on the business logic within individual services rather than the intricacies of network management.

TABLE 2. Comparison of observability tools [64].

Tool [57]	Open Source	Proprietary Version	Supported Platforms	Key Strengths	Lacking
Prometheus	Yes	No	Linux, Windows, macOS	Reliability and scalability	No tracing or logs support
Datadog	No	Yes	Linux, Windows, macOS	Real-time performance dashboards	High cost
Grafana	Yes	Yes (Grafana Enterprise)	Linux, Windows, macOS	Sophisticated visualization capabilities	No native data collection
Zipkin	Yes	No	Cross-platform (Java-based)	Tracing and diagnosing latency issues	No metrics or logs support
Weave Scope	Yes	No	Linux	Container monitoring and visualization	No tracing
Loki	Yes	No	Linux, Windows, macOS	Cost-effective log management	Logs only
OpenTelemetry	Yes	No	Cross-platform (language dependent)	Comprehensive observability framework	Requires storage
BPFTrace	Yes	No	Linux (requires BPF)	Dynamic tracing and real-time performance analysis	Low-level observability
Istio	Yes	No	Linux, Windows	Real-time traffic management and observability	High deployment complexity
Jaeger	Yes	No	Linux, Windows	Deep insights into service interactions and performance	Tracing only

This taxonomy of microservices observability is structured into six interrelated categories, each serving distinct monitoring objectives. The Purpose of Monitoring includes performance analysis, failure root cause analysis, anomaly detection, and overall application health, which all aim to ensure system reliability and efficiency. These purposes are supported by specific Monitoring Parameters, such as logs for understanding system events, metrics for performance evaluation, and traces for tracking requests across services. Monitoring occurs at different Levels, including the network, system, and services layers, each offering insights into various aspects of the microservices architecture. The Implementation Paradigm covers deployment environments like cloud, fog, and edge, influencing how monitoring tools are deployed. The Microservices Deployment Platforms, such as Kubernetes or Docker Swarm, integrate with these paradigms to manage and orchestrate services. Finally, various Monitoring Tools like Prometheus, Loki, and Grafana provide the technical capability to observe and manage microservices, while specific Architecture Patterns like service mesh and API gateways ensure that the monitoring frameworks are well-integrated with the underlying service architecture.

V. REVIEW AND COMPARATIVE ANALYSIS OF STATE-OF-ART MICROSERVICES OBSERVABILITY SOLUTIONS

The section consists of two parts. The first part offers a detailed review of the state-of-the-art microservices observability solutions, highlighting the most recently published research findings. In the second part, a comparative analysis of these solutions is provided based on the thematic taxonomy defined in section IV.

A. STATE-OF-THE-ART MICROSERVICES OBSERVABILITY SOLUTIONS

This subsection highlights recently published research works and presents a comprehensive review of state-of-the-art microservices observability solutions. In recent years, observability has become a critical focus in microservices architecture due to the complexity introduced by distributed systems. Modern research emphasizes the necessity of observability in ensuring service reliability, performance monitoring, and effective troubleshooting. Recent studies have focused on advanced techniques for tracing, logging, and metrics collection to enhance visibility and understanding of microservices environments.

1) MDF (MODEL DRIVEN FRAMEWORK)

Fè et al., [65] proposed a model-driven methodology to evaluate the dependability and energy consumption of Kubernetes-based cloud fog systems. The proposed model uses a stochastic approach to predict system behavior, with a particular focus on the performance of cloud and fog computing layers under different configurations. The proposed system monitored system availability and energy consumption, with a significant finding demonstrating a 22.33% increase in system availability and a minimal 1.33% increase in energy consumption. This model offers a structured approach to analyzing and comparing the dependability of different systems and serves as a guide to optimize source allocation. The proposed methodology involves empirical validation through rigorous testing in controlled environments to ensure accurate dependability and power consumption assessments, contributing to enhanced system performance and sustainability in cloud fog computing environments.

2) METROFUNNEL

Cinque et al., [66] proposed a monitoring system for microservices using event logs combined with passive black box tracing to capture data like resource usage and operational statistics essential for maintaining service continuity. The proposed method, MetroFunnel, enhances troubleshooting by automatically generating detailed execution traces without modifying the microservices, making it suitable for complex environments. Tested in Docker and Kubernetes setups, MetroFunnel supports efficient problem-solving with minimal performance and data overhead, proving an effective tool for operational transparency and troubleshooting in microservices architectures.

3) MOA (MODERN OBSERVABILITY APPROACH)

Tzanettis et al., [67] focused on enhancing the observability and management of distributed applications, particularly those based on microservices, by addressing the disjoint nature of information provided by existing frameworks and cloud computing orchestration platforms. The proposed methodology integrates various observability signals such as metrics, logs, and distributed tracing mechanisms from multiple open-source frameworks to form a unified view that facilitates performance issue insights and root cause analysis. This approach is demonstrated through a pilot implementation involving the stress testing of a proof-of-concept microservices-based application, which validated the methodology by identifying main latency causes and enhancing the application's behavior understanding under different conditions. The results confirmed the effectiveness of the data fusion approach in providing deeper and more actionable observability of distributed applications, thereby supporting better management and troubleshooting of these systems.

4) AET (APPROACH FOR EXPLAINABILITY AND TRACEABILITY)

Pierre-Jean and Zdun [68] proposed a method to analyze and recover the architecture of complex microservices systems, focusing on security, explainability, and traceability. The approach consists of three main steps: abstracting tree structures, exploring using generic detectors, and scanning to create a linear view that outlines the system's architecture. The study tested this method on the Piggymetrics application, showing that it could accurately identify the architecture components and security annotations found in manual analyses. This demonstrates the method's precision and ability to link findings to specific code locations. However, the study also revealed some challenges in detecting all security features due to their scattered nature across different code sections, pointing out opportunities for further refining the detectors and enhancing the overall approach.

5) SuanMing

Grohmann, et al., [69] proposed a framework to predict the root cause analyses of the microservices performance degradation over the cloud infrastructure. The proposed system is based on the machine learning model, is deployed over the virtual machine that can monitor the traces data for the fine-grain analysis of the system and predict the microservices performance degradation 100s to 200s ahead of it happens. The accuracy of the proposed system is 90% and its F1 score is 0.7 with minimal overheads.

6) BbMA

Brondolin and Santambrogio [70] proposed an approach for microservices monitoring at runtime with less overhead using eBPF technology. This approach is integrated with the existing tool Sysdig for CPU, memory, network IO metrics, Weave scope for CPU usage and network connections, and Prometheus for APIs, time-series database, and visualization interface.

7) NEZHA

Yu et al., [71] proposed an observability framework for the root cause analyses of the microservices-based system. The collected data was multi-modal and contained the heterogeneous data of logs, traces, and metrics at the services level and inner services level, and the varied data into a homogeneous form for further analyses to better address the root cause in case of service, failure. This approach mainly focuses on anomaly detection, however, the suggested approach is unable to detect certain byzantine errors, such as providing the user with an irrational result. Since the proposed approach can process multi-modal data and identify a broader range of fault behaviors, it outperforms all baseline techniques at the service level with high accuracy (97% at both top 3 and top 5 and 90% at top 1), making it the most effective tool available.

8) KUNERVA

Lee and Nam [72] proposed a framework based on network policy generation to avoid security attacks occurring due to misconfiguration. They applied Cilium Hubble for the network monitoring to collect the logs that contain less data on source and destination pods' name and their port numbers. They collected logs related to the system using eBPF and LSM from various Levels from 3-7 and found policies. Except for the log collector, which operated with the microservice containers on different machines, every component of the proposed system was configured on the main computer. Throughout the tests, they looked for policies every 10 seconds and monitored the CPU and memory usage every five minutes.

9) CdOF

Ranjitha et al., [73] proposed a framework for the observability of the microservices at the network and host domain for better performance evaluation and analysis of the actual cause of the microservices failure. Multiple domain observability provides the fine grain end-to-end from the sender to the host observability of the system. The observability framework is

based on eBPF while imposing less overheads of 2.6% of CPU usage. This system correlates the internal application traffic (within pod spaces) with the broader external network activities (across the physical network IP space) for the precise identification and diagnosis of the sources and reasons for network performance issues.

10) AdOB

Monteiro et al., [74] proposed an observability approach based on game theory a statistical approach for forensic-ready microservices systems to investigate efficiently in case of any security breach and performance failures. The researcher collects logs, traces, and metrics while considering the user (malicious or legitimate) and microservices as game players, and collects the data in case of unusual activities done by the malicious user as defined in the game theory strategies and maintains the related log. The findings show that adaptive observability outperforms observability by a significant margin, it improves detection rates by 26.97% to 42.50% in sampling observability and by 3.1% to 38.96% in full observability.

11) eBPFM

Wustrich et al. [75] proposed an eBPF-based matcher that monitors and logs the communication patterns of applications, linking network traffic with their respective applications. eBPF matcher collects the data related to the ingress and egress packets containing packet information, associated process ID, and thread group and matches them with their specific application to identify a malicious network flow. For this purpose, the researchers deployed a botnet to perform an intrusion, and the proposed tool maintained logs and successfully identified the botnet's abnormal activity. Moreover, the researchers gathered relevant data while minimizing data redundancy, which significantly reduced the overhead of the eBPF-based matcher.

12) DESK

Usman et al., [76] proposed an observability framework named microservices-based DESK for fine-grain analysis at the services level of the microservices-based containerized environment. The proposed framework is designed to efficiently manage and control Kubernetes-orchestrated edge clusters. It comprises six stages including Measurement, Fusion, Storage, Visualization and Notification, and Provisioning and Orchestration (P&O), as a separate loosely coupled component. Granulated metrics, logs, and traces are collected from various points using lightweight agents like Telegraf, Promtail, and Jaeger in the measurement stage. Prometheus, Loki, and Elasticsearch are utilized for real-time observability, data access, and analysis. Grafana is employed for data visualization and Prometheus for open-access anomaly detection and threshold-based alerts management. Provisioning and Orchestration (P&O) is done by Helm and Kubectl tools for edge scalability and operations management. Based on the results, the DESK utilizes approximately 2.5% of the total CPU and memory available on the hardware. Despite this low overhead, it demonstrates rapid troubleshooting and resolution of diverse operational issues arising from varied observability data. The framework is mostly based on existing open-access tools like Prometheus, Grafana, Telegraf, and others.

13) OESS

Alanezi and Mishra [77] design, implement, and evaluate the performance of a microservices-oriented architecture for an edge server. It further illustrates its practicality by building comprehensive IoT applications from start to finish. According to the research, monitoring is a crucial component of the microservices architecture. The system includes a Microservices Registry that tracks the status of each microservices, determining which are active and which are not. This information is sent to the Orchestrator, which uses it to create launch configurations for new services or applications. The Orchestrator's configurations are then sent to the Main Controller, which launches the appropriate Application Controller for the new application and provides the client with the necessary port number. This architecture heavily utilizes Docker for containerization, enabling the virtualization of each microservices and allowing for comprehensive performance evaluation, including the monitoring of inter-microservices communication.

14) MSM

Zdun et al. [78] addressed the challenges of microservices architecture for security, considering the size, complexity, and continuous evolution of these systems. This method employed three commonly used security strategies: secure communication, identity management, and observability. They emphasized the importance of three key observability techniques in microservices architecture: Logging, monitoring, and tracing. Logging records system events to identify defects and conduct audits for each microservice. Monitoring involves the use of metrics to assess the service request management to provide a holistic view of the system's overall performance. Tracing or distributed tracing to track individual transactions through the various services in the system, focusing on performance and cross-service optimizations. The researcher developed an innovative approach to evaluate compliance with architectural design decisions (ADDs) in microservices security using specialized metrics. In this method, they utilized regression analysis, a technique that effectively and accurately correlates the dependent and independent variables.

15) KAIJU

The Kaiju Project [79] introduced an event-driven approach for enhancing observability in complex microservices architectures, focusing on integrating monitoring of metrics, logs, and traces. By treating these data sources as events, the project aims to overcome the limitations of traditional monitoring methods in dynamic environments. This integration

allows for near real-time analysis of system behavior, enabling faster and more effective responses to operational issues. Implemented and tested in a real-world setting with a software company's deployment, the Kaiju system demonstrates its ability to provide detailed, real-time insights into system performance and health efficiently, without demanding excessive resources. The results underline the effectiveness of the Kaiju system as a potent tool for observability in modern distributed systems.

16) STEAM

He et al., [80] proposed a framework STEAM, approach that focuses on enhancing observability in distributed systems by addressing the limitations of traditional uniform sampling methods. The STEAM methodology employs a novel trace sampling strategy that preserves a wide range of trace types and maximizes the diversity of the sampled data. This is achieved by using a Graph Neural Network (GNN) for trace representation, which incorporates domain-specific knowledge to better capture the nuances of trace data, and employing Determinantal Point Processes (DPP) to focus on sampling mutually dissimilar traces. These techniques together ensure that the observability of the system is maintained, enabling better system diagnostics and performance optimization.

17) AAD

Mart et al., [81] focused on enhancing observability and anomaly detection within Kubernetes clusters. The proposed methodology involves using Prometheus metrics for continuous monitoring and applying forecasting techniques to predict potential anomalies. By forecasting metrics, the system can proactively alert users to anomalies, allowing for quicker response times and potentially avoiding system defects. The results demonstrate that this method can effectively enhance the observability of the Kubernetes clusters, providing a more automated and predictive monitoring solution.

18) CHAOSORCA

Simonsson et al., [82] presented CHAOSORCA, a novel fault injection system designed to assess the resilience of containerized applications against system call errors without requiring modifications to the application within Docker-based microservices. The system is implemented in real conditions without altering the application, using a combination of monitoring different levels of call to the system and implementing perturbations. The methodology of CHAOSORCA includes three main components: a monitor, a perturbator, and an orchestrator. The monitor tracks system metrics and system calls, the perturbator introduces specific disruptions into the system calls, and the orchestrator manages the experiments, recording the impact of perturbations on system behavior. CHAOSORCA was evaluated on three real-world applications: a file transfer client, a reverse proxy server, and a microservice-oriented web application. The results demonstrate CHAOSORCA's effectiveness in identifying weaknesses in resilience mechanisms related to system calls.

19) eBPFNM

The paper [83] developed an event-driven network monitoring platform using an extended Berkeley Packet Filter (eBPF) to improve microservices observability. It addressed key challenges such as acquiring customized monitoring metrics, dynamic load balancing, optimizing resource utilization, and implementing rate limiting to handle traffic surges. The system leveraged eBPF for real-time, low-overhead monitoring of network and system performance, providing detailed insights into microservices interactions. Additionally, it incorporated load-balancing strategies to optimize service performance and resource management techniques to enhance CPU and memory efficiency. By integrating the platform with Kubernetes, scalability, and reliability were ensured, making it well-suited for cloud-based microservices environments.

20) DyCause

Pan et al., [84] developed DyCause, a diagnostic tool for microservices systems that operate from user space and use crowd crowd-sourcing to collect diagnostic data, avoiding the need for kernel-side access. Unlike traditional methods that depend on kernel monitoring, DyCause uses lightweight API log sharing and a statistical algorithm to uncover time-varying causal relationships between services. This approach constructs a dynamic map of service interactions, necessary for tracing failure propagation. Tested in simulated and real-world cloud environments, DyCause outperformed conventional kernel-based methods in accuracy, efficiency, and data sensitivity, proving effective in identifying root causes of failure with minimal data. This makes DyCause a viable solution for managing complex, dynamic microservices architectures.

21) ADMBA

Vaughan et al., [85] investigated the high availability (HA) of Kubernetes in a private cloud setting by testing its default configurations against various failure scenarios. The experiments, which utilized VLC video streaming microservices, were designed to monitor Kubernetes' response to both pod and node failures, assessing key metrics such as reaction, repair, recovery, and outage times. The results indicated that while Kubernetes efficiently handles internal-triggered failures through administrative commands, it struggles with extended downtime when dealing with external failure triggers, revealing a gap in achieving HA under default settings. The study underscores the necessity for optimizing Kubernetes configurations to enhance responsiveness and reliability in real-world operational scenarios.

22) eBPFOaS

Liu et al. [86] proposed a protocol-independent non-intrusive method to analyze network observability in container

networks using eBPF technology within Kubernetes clusters. The methodology consists of three main components: transparent data collection without requiring code modifications, data aggregation into meaningful metrics, and intelligent analysis using machine learning to detect anomalies and diagnose network performance issues. The evaluation results demonstrate minimal impact on system performance, with less than a 1.2% increase in latency and under 1% impact on throughput, while effectively detecting network anomalies and finding specific instances of network problems, such as significant delays and packet losses. This system allows for real-time monitoring and problem-solving in complex containerized environments, highlighting its efficiency and minimal operational intrusion.

23) ENMA

This research study [87] introduces an advanced network monitoring framework that leverages Extended Berkeley Packet Filter (eBPF) and Express Data Path (XDP) to optimize network analytics efficiency. This framework utilizes a shared monitoring primitive, which can be partially offloaded to a SmartNIC, to calculate essential high-level traffic metrics. These metrics are used to selectively trigger detailed analyses when necessary, effectively minimizing redundant processing. Tested on a Netronome, NFP, SmartNIC, and the Linux kernel, the implementation significantly reduces resource utilization by dynamically managing application executions based on the current network conditions, greatly enhancing the scalability and effectiveness of network monitoring operations.

24) eBPFTT

The paper [88] explores the application of the extended Berkeley Packet Filter (eBPF) for non-intrusive monitoring of containerized user-space applications in production environments, especially using Linux subsystems. The research centers on monitoring Interledger connectors through two comprehensive implementations of the Interledger protocol. Using eBPF, the authors track and analyze critical performance metrics like CPU utilization, memory, and network traffic without disrupting the running applications. The findings highlight eBPF's utility in providing in-depth system insights, vital for identifying performance bottlenecks and enhancing application efficiency in a non-disruptive manner. This paper emphasizes the value of eBPF in improving system observability, specifically beneficial in scenarios where conventional monitoring tools fall short or are intrusive.

25) NETOBSERV

The paper [89] presents the netobserv-ebpf-agent, an optimized network observability agent for monitoring cloud-based microservices using eBPF technology. This tool operates smoothly across various hosting environments and is independent of underlying network configurations, significantly reducing both CPU and memory usage while meeting stringent performance and reliability standards. Utilizing diverse eBPF maps, the agent efficiently collects and processes network traffic data, showing marked performance improvements compared to existing solutions through detailed benchmarks. Now available as an open-source, netobserv-ebpf-agent has been successfully incorporated into the Red Hat OpenShift Container Platform, illustrating its practical benefits and effectiveness in improving network observability for cloud applications.

26) ADAM

The researcher introduced a novel technique [90] for detecting anomalies in microservice architectures by monitoring distributed traces and profiling metrics. The authors developed a model, structured as an annotated directed acyclic graph (DAG), which utilized six essential profiling metrics to thoroughly characterize standard system operations. This model effectively captured both linear and non-linear metric relationships, enhancing its ability to detect anomalies. The effectiveness of the method was validated on two open-source benchmarks, achieving an F1 score of up to 86%. Additionally, this approach supported developers in conducting root cause analysis by comparing the anomalous behavior with the baseline behavior established by the model.

27) OTHERS OBSERVABILITY SOLUTIONS

We explore additional observability solutions of a microservices-based containerized platform that performed the root cause analysis in case of service failure and various performance parameters. However, they did not align with the holistic approach of our taxonomy. They either propose frameworks or methodologies that are still experimental or lack comprehensive quantitative evaluations.

Shiraishi et al. [91] proposed the extended Berkeley Packet Filter (eBPF) technology for real-time monitoring, as a sensor for capturing container network metrics. The adoption of microservices architecture has become increasingly prevalent within large-scale web services, driven by the need for more flexible and scalable development and deployment solutions. This shift towards microservices is often accompanied by the adoption of container technologies such as Docker, which provides lightweight and portable environments for running microservices. While microservices offer numerous benefits, they also introduce challenges in monitoring and managing the underlying infrastructure to ensure service quality and reliability. Effective monitoring of microservices requires not only tracking the performance and behavior of individual services but also monitoring the infrastructure on which these services run. This includes containers, virtual machines, and physical machines. Without comprehensive visibility into the infrastructure, it becomes difficult to identify and troubleshoot issues that may impact service quality. Therefore, there is a growing need for monitoring solutions that provide insights into both the services and the supporting infrastructure.

ViperProbe [92] is an eBPF-based microservices collection framework offering two key features: dynamic sampling and collection of diverse system metrics. Dynamic sampling allows for adaptability in monitoring, while diverse system metrics ensure a comprehensive view of microservice health and performance. ViperProbe leverages eBPF (extended Berkeley Packet Filter) technology to collect metrics. It takes advantage of common design patterns, such as service mesh, for offline analysis to optimize the set of collected metrics before deployment in production environments. Further, it is the first scalable eBPF-based dynamic and adaptive microservices metrics collection framework. It suggests that ViperProbe offers limited overhead while significantly enhancing traditional management tasks such as horizontal auto-scaling. The proposed system demonstrates effectiveness in addressing traditional management tasks, suggesting improved efficiency and effectiveness in monitoring microservices. However, specific quantitative results regarding performance overhead or comparative analysis with existing tools are not provided in this excerpt. They aim to alleviate the challenges of monitoring microservices by providing a dynamic and adaptive approach to metrics collection, leveraging eBPF technology and offline analysis of common design patterns. It suggests potential benefits in terms of efficiency and effectiveness in managing microservice architectures.

Multi-Layer Observability for Fault Localization, this research [93] is a proposed solution to enhance microservices observability by addressing the challenge of correlating logs across various layers of the application stack. They suggest that existing monitoring practices often concentrate solely on generating logs, metrics, and traces at the application layer. However, for effective fault localization, it's crucial to correlate logs across non-application layers like load balancers and databases. The researchers propose an observability library and platform aimed at addressing this challenge. The observability library likely provides tools and utilities to tag logs with a common request identifier, enabling correlation across different layers. Meanwhile, the observability platform serves as the central hub for collecting, analyzing, and visualizing these correlated logs. The proposed solution emphasizes the importance of tagging logs with a common request identifier to facilitate correlation across different layers of the microservices architecture. This allows for a holistic view of the entire request life-cycle, from the front-end load balancer to the back-end database. Further, the observability platform is designed in a modular and extensible manner, enabling it to accommodate multiple types of errors and issues beyond the initial scope. This suggests that the platform is adaptable to evolving requirements and can scale with the growing complexity of microservices environments. The proposed observability platform has been tested on five open-source benchmarks. The results indicate that the tool can reliably and precisely detect elusive issues, implying its effectiveness in real-world scenarios. Overall, this paper proposes a comprehensive observability solution for microservices architectures, addressing the challenge of correlating logs across different layers. By leveraging a common request identifier and a modular observability platform, the proposed solution aims to enable precise fault localization and detection of issues, ultimately enhancing the reliability and performance of microservices-based applications. Multi-Layer Observability for Fault Localization, the research [94] outlines the shift from standalone service-oriented architecture (SOA) to microservices architecture, driven by the need for handling larger customer bases and making systems more responsive to changes. Microservices architecture involves developing and maintaining each functionality as a separate service, offering benefits like scalability and ease of deployment. The researchers aim to address the limitations of existing monitoring solutions, which often focus only on the application layer. The proposed mechanism offers a comprehensive view of the microservices architecture, facilitating a better understanding and management of the system. The researchers used distributed tracing frameworks such as Jaeger or Zipkin to implement distributed tracing. These tools enable capturing and correlating traces across microservices. Process mining techniques involve analyzing event logs to discover process models and patterns, which can be visualized for monitoring purposes. The results of this approach would include improved observability of the microservices architecture, better understanding of system behavior, and quicker detection of issues or bottlenecks. However, it's important to note that implementing such a monitoring mechanism may introduce performance overhead, especially in terms of increased network traffic for distributed tracing and computational resources for process mining. Furthermore, the strategies are discussed in the paper for mitigating this overhead and optimizing the monitoring mechanism for production environments in the future. A Real-time, Scalable Monitoring and Analytics, the research [95] highlights the complexity of monitoring distributed microservices-based software applications, particularly in environments where large amounts of user and transaction data are generated at a rapid pace. Real-time monitoring is crucial for gaining insights into user interactions and transactions and facilitating decisions for stakeholders. The focus is on capturing data effectively to learn application usage patterns and predict events such as error types using machine learning methods. The researchers utilized various tools and techniques for monitoring microservices and analyzing user interaction and transaction data. They captured event data from execution logs of software applications running on distributed microservices. This could involve using logging frameworks such as Logback, Log4j, or Fluentd to collect logs from each microservice. Machine learning methods were employed to predict usage patterns and identify error types in real-time to predict error types based on historical data. The system was capable of monitoring user actions and transactions in real-time, enabling immediate response to events and issues. This real-time monitoring capability is essential for maintaining

system health and ensuring a positive user experience. The researchers likely conducted performance tests to evaluate the system's capacity to handle growing data volumes and maintain responsiveness.

Adaptive Containerization for Microservices, the research [96]addresses microservices observability by focusing on resource allocation and performance monitoring within containerized microservice deployments. To achieve this, the researchers likely utilize a combination of tools and techniques for monitoring the behavior and performance of microservices. Techniques such as logging, tracing, and monitoring are employed to gather data on resource utilization, response times, and overall system performance. Additionally, the framework may incorporate built-in observability features provided by container orchestration platforms like Kubernetes, enabling real-time monitoring and analysis of microservice performance. Performance evaluation metrics provide quantifiable evidence of the framework's impact on microservice deployments. Regarding performance overhead, the research discusses the computational overhead introduced by the resource allocation framework and any trade-offs made to achieve cost optimization and service guarantees. Strategies for mitigating performance overhead, such as fine-tuning resource allocation algorithms or optimizing container configurations, also be discussed.

B. COMPARATIVE ANALYSIS OF STATE-OF-THE-ART MICROSERVICES OBSERVABILITY SOLUTIONS

This subsection conducts a critical analysis of microservices observability solutions, building on the thematic taxonomy outlined in Section IV. Table 3 presents an overview of this comparison and offers a structured view of the current state-of-the-art microservices observability solutions.

Observability is crucial for monitoring and analyzing the behavior and performance of complex systems. It enables engineers to understand, troubleshoot, and optimize system operations effectively. This section includes the comparative analyses of the state-of-the-art microservices observability frameworks. In comparing the various proposed methods for observability, it's evident that each framework has its focus and priorities regarding different observability levels within a system: System Level, Services Level, and Network Level. System-level observability is important because it provides insights into the functioning of the system architecture and infrastructure, allowing engineers to identify potential issues, optimize resource allocation, and ensure system reliability and performance. Methods such as MOA [67], AET [68], BBMA [70], Nezha [71], KUNERVA [72], STEAM [80], AAD [81], CHAOSORCA [82], DyCause [84], eBPFTT [88], and ADAM [90] prioritize observability at the system level. These frameworks emphasize monitoring and understanding the overall behavior and performance of the system as a whole, including factors such as resource utilization, system health, and stability. They aim to provide insights into the functioning of the system architecture and infrastructure. Services level observability focuses on monitoring and analyzing the behavior and performance of individual services within the system. MDF [65], Metrofunnel [66], SuanMing [69], ADMBA [85], Kaij [79], and MSM [78] focus on observability at the services level. These frameworks prioritize monitoring and analyzing the behavior and performance of individual services within the system. They aim to provide detailed insights into service interactions, dependencies, and performance metrics to ensure the reliability and efficiency of service-based architectures. Network-level observability involves monitoring and analyzing network traffic, communication patterns, and performance metrics to ensure the reliability, security, and efficiency of network infrastructures. Frameworks like CDOF [73], eBPFOAS [86], ENMA [87], and netobserv [89] are centered around observability at the network level. These frameworks emphasize monitoring and analyzing network traffic, communication patterns, and performance metrics to ensure the reliability, security, and efficiency of network infrastructures. They aim to provide insights into network behavior, traffic patterns, and potential bottlenecks or security threats.

In the realm of edge computing, the observability of systems is paramount for ensuring efficient operations and troubleshooting potential issues. A comparative analysis of various proposed methods reveals the types of data each framework observes for achieving observability. Frameworks such as MDF [65], MOA [67], BBMA [70], Nezha [71], CDOF [73], ADOB [74], DESK, MSM [78], Kaiju [79], AAD [81], CHAOSORCA [82], DyCause [84], eBPFOAS [86], and ADAM [90] prioritize the observation of logs for edge computing observability. Logs provide a detailed record of system events, errors, and activities, facilitating effective troubleshooting and analysis. Others such as MDF [65], MOA [67], BBMA [70], Nezha [71], CDOF [73], ADOB [74], DESK [76], Kaiju [79], AAD [81], CHAOSORCA [82], DyCause [84], OAS, and ADAM [90] are among the frameworks that collect metrics data for observability. Metrics offer insights into system performance, resource utilization, and other key parameters, aiding in performance analysis and optimization. While several frameworks, including Metrofunnel [66], MOA [67], AET [68], SuanMing [69], BBMA [70], Nezha [71], CDOF [73], ADOB [74], DESK [76], Kaiju [79], CHAOSORCA [82], DyCause [84], eBPFOAS [86], and eBPFTT [88], prioritize tracing data for observability. Traces provide visibility into the flow of requests and transactions across distributed systems, enabling the identification of performance bottlenecks and latency issues. Some frameworks adopt a mixed approach to observability, encompassing a combination of logs, metrics, and traces. Examples include MDF [65], MOA [67], BBMA [70], Nezha [71], CDOF [73], ADOB [74], DESK [76], Kaiju [79], AAD [81], CHAOSORCA [82], DyCause [84], OAS, and ADAM [90], which leverage multiple data types to provide comprehensive insights into system behavior and performance. Frameworks like

TABLE 3. Comparison of state-of-the-art microservices observability frameworks.

FrameWork	Monitoring Scope			Monitoring Parameters			Implementation Layers			Evaluation Metrics				Deployment Environ-ment		Monitoring Tools.										Architecture Pattern		
	Sys	Ser	NW	Log	Mt	Tr	Cl	Fg	Eg	PA	RCA	AD	AH	DS	Kub	Pr	Dl	Gr	Zp	WS	Lo	OT	Is	Jg	BPFT	SM	AG	ED
[65]	×	✓	×	✓	✓	✓	✓	✓	×	✓	✓	✓	✓	×	✓	✓	×	✓	×	×	×	×	×	×	×	×	×	×
[66]	×	✓	×	×	×	✓	✓	×	✓	✓	✓	×	✓	×	✓	×	×	×	✓	×	×	×	×	×	×	×	×	×
[67]	✓	×	×	✓	✓	✓	✓	×	×	✓	✓	×	✓	×	✓	✓	×	✓	✓	×	×	✓	×	×	×	×	×	×
[68]	✓	×	×	×	×	✓	✓	×	×	✓	✓	✓	✓	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×
[69]	×	✓	×	×	×	✓	✓	×	×	✓	✓	×	×	×	×	×	×	×	✓	×	×	×	×	✓	×	×	×	×
[70]	✓	×	×	✓	✓	✓	✓	×	×	✓	✓	×	✓	✓	✓	✓	✓	✓	✓	✓	×	×	×	×	×	×	✓	×
[71]	✓	×	×	✓	✓	✓	✓	×	×	×	✓	×	×	×	✓	✓	×	✓	×	×	✓	✓	×	×	×	NA	NA	NA
[72]	✓	×	×	✓	×	×	✓	×	×	✓	✓	✓	×	✓	✓	✓	×	✓	×	×	×	×	×	×	×	×	×	×
[73]	×	×	✓	✓	✓	✓	✓	×	×	✓	✓	✓	×	×	✓	×	×	×	×	×	×	×	×	×	×	✓	×	×
[74]	×	✓	×	✓	✓	✓	✓	×	×	✓	✓	×	✓	✓	✓	×	×	×	×	×	×	×	×	×	×	×	×	×
[75]	✓	×	×	✓	×	×	×	×	×	✓	✓	✓	×	Na	Na	×	×	×	×	×	×	×	×	×	✓	×	×	×
[76]	×	✓	×	✓	✓	✓	×	×	✓	✓	✓	✓	×	×	✓	✓	×	✓	×	×	✓	✓	×	×	×	×	×	×
[77]	×	✓	×	×	✓	×	×	×	✓	✓	✓	×	✓	✓	×	×	×	×	×	×	×	×	×	×	×	×	×	×
[78]	×	✓	×	✓	×	✓	✓	×	✓	✓	✓	✓	×	NA	NA	×	×	×	×	×	×	×	×	×	×	×	✓	×
[79]	✓	×	×	✓	✓	✓	✓	×	×	✓	✓	✓	✓	×	✓	×	×	×	✓	×	×	✓	×	✓	×	×	×	✓
[80]	✓	×	×	×	×	✓	×	×	×	✓	✓	×	✓	×	×	×	×	×	×	×	×	✓	×	×	×	×	×	×
[81]	✓	×	×	✓	✓	✓	✓	×	×	✓	✓	✓	×	×	✓	✓	×	×	×	×	×	×	×	×	×	×	×	×
[82]	✓	×	×	✓	✓	✓	×	×	×	✓	✓	✓	×	✓	✓	×	×	×	✓	×	×	×	×	✓	×	×	×	×
[83]	×	×	✓	×	✓	×	×	×	×	✓	✓	×	✓	×	✓	✓	×	✓	×	×	×	×	×	×	✓	×	×	✓
[84]	✓	×	×	✓	✓	✓	✓	×	×	✓	✓	×	✓	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×
[85]	×	✓	×	×	✓	×	✓	×	×	×	✓	×	×	×	✓	✓	×	✓	×	×	×	×	×	×	×	×	×	×
[86]	×	×	✓	✓	✓	✓	✓	×	×	✓	✓	×	×	×	✓	×	×	×	×	×	×	×	×	×	✓	×	×	×
[87]	✓	×	✓	×	✓	×	×	×	✓	✓	×	×	✓	NA	NA	×	×	×	×	×	×	×	✓	×	✓	NA	NA	NA
[88]	✓	×	×	×	×	✓	✓	×	×	✓	×	✓	×	×	×	×	×	×	×	×	×	×	×	×	✓	×	×	×
[89]	×	×	✓	×	✓	×	✓	×	×	✓	✓	×	×	×	×	×	×	×	×	×	×	×	×	×	✓	×	×	×
[90]	✓	×	×	✓	✓	✓	×	×	×	×	×	✓	×	×	✓	✓	×	×	×	×	×	×	✓	✓	×	✓	×	×

Abbreviations and Symbols: Sys: System Level, Ser: Services Level, NW: Network, Log: Logging, Mt: Metrics, Tr: Traces, Cl: Cloud, Fg: Fog, Eg: Edge, PA: Performance Analysis, RCA: Root Cause Analysis, AD: Anomaly Detection, AH: Application Health, DS: Docker Swarm, Kub: Kubernetes, Pr: Prometheus, Dl: Datadog, Gr: Grafana, Zp: Zipkin, WS: WeaveScope, Lo: Loki, OT: OpenTelemetry, Is: Istio, Jg: Jaeger, BPFT: BPFTrace, SM: Service Mesh, AG: API Gateway, ED: Event-Driven. NA: Not Available, Na: Not Applicable.

Metrofunnel [66], AET [68], SuanMing [69], STEAM [80], ADMBA [85], ENMA [87], eBPFTT [88], and netobserv [89] have more limited observability capabilities, focusing primarily on tracing or lacking support for certain data types like logs or metrics.

However, the choice of observability framework for edge computing depends on the specific data requirements and objectives of the system or application being monitored. Each framework offers distinct capabilities for observing logs, metrics, and traces, allowing stakeholders to select the most suitable approach based on their needs. When analyzing the implementation paradigms of various proposed frameworks for edge computing, it's evident that each framework has its approach and priorities regarding deployment across different environments: Cloud, Fog, and Edge. Frameworks like MDF [65], Metrofunnel [66], AET [68], SuanMing [69], BBMA [70], Nezha [71], KUNERVA [72], CDOF [73], ADOB [74], eBPFM [75], MSM [78], STEAM [80], CHAOSORCA [82], eBPFNM [83], and ADAM [90] support deployment in cloud environments. These frameworks are designed to operate seamlessly within cloud infrastructures, leveraging the scalability and resources offered by cloud platforms. Only a few frameworks, such as DESK [76], OESS [77], Kaiju [79], AAD [81], DyCause [84], and ADMBA [85], are tailored for deployment in fog environments. Fog computing involves extending cloud computing capabilities to the edge of the network, closer to where data is generated and consumed. These frameworks are optimized for use in fog environments, offering efficient processing and analysis of data at the edge. Some frameworks, including DESK [76], OESS [77], Kaiju [79], AAD [81], DyCause [84], ADMBA [85], eBPFOAS [86], ENMA [87], eBPFTT [88], netobserv [89], and ADAM [90], are specifically designed for deployment at the edge. Edge computing involves processing data locally at the edge of the network, closer to the data source, to reduce latency and bandwidth usage. These frameworks are tailored to address the unique challenges and requirements of edge deployments, such as limited resources and intermittent connectivity. Overall, the choice of a specific deployment paradigm depends on factors such as the intended use case, data processing requirements, and the availability of resources. While cloud deployment offers scalability and centralized management, fog, and edge deployments provide low-latency processing and support for edge computing scenarios. Each framework caters to different deployment environments, allowing organizations to select the most suitable option based on their specific needs and objectives in edge computing applications.

When examining the purpose of monitoring across various proposed frameworks for edge computing, it becomes clear that each framework has distinct objectives and priorities concerning different aspects of monitoring: performance analysis, root cause analysis (RCA), anomaly detection, and application health monitoring. Most frameworks, including MDF [65], Metrofunnel [66], MOA [67], AET [68], SuanMing [69], BBMA [70], KUNERVA [72], CDOF [73], ADOB [74], eBPFM [75], DESK [76], OESS [77], MSM [78], Kaiju [79], STEAM [80], AAD [81], CHAOSORCA [82], eBPFNM [83], DyCause [84], eBPFOAS [86], eBPFTT [88], and netobserv [89], prioritize performance analysis as a key objective of monitoring. These frameworks aim to assess system performance metrics, identify bottlenecks, and optimize resource utilization for enhanced efficiency and reliability. Several frameworks, including MDF [65], Metrofunnel [66], MOA [67], AET [68], SuanMing [69], BBMA [70], KUNERVA [72], CDOF [73], ADOB [74], eBPFM [75], DESK [76], OESS [77], MSM [78], Kaiju [79], STEAM [80], AAD [81], CHAOSORCA [82], DyCause [84], and eBPFOAS [86], prioritize RCA as part of their monitoring objectives. These frameworks aim to identify the underlying causes of system issues or failures, enabling timely resolution and preventing recurrence. Anomaly detection is emphasized by frameworks like MDF [65], AET [68], BBMA [70], and Kaiju [79]. These frameworks aim to detect abnormal behavior or deviations from expected patterns within the system, facilitating early identification of potential issues or security threats. Monitoring application health is a focus for frameworks such as MDF [65], Metrofunnel [66], ADOB [74], DESK [76], OESS [77], Kaiju [79], STEAM [80], AAD [81], CHAOSORCA [82], DyCause [84], eBPFOAS [86], and netobserv [89]. These frameworks prioritize monitoring the health and status of applications deployed in the edge environment, ensuring their availability, reliability, and performance. The choice of monitoring framework depends on the specific monitoring objectives and requirements of the edge computing application. While some frameworks prioritize comprehensive performance analysis and RCA, others focus on specific aspects such as anomaly detection or application health monitoring. Each framework offers unique capabilities tailored to address different aspects of monitoring in edge computing environments.

When considering the deployment platforms supported by various proposed frameworks for edge computing, it's evident that each framework has distinct capabilities and compatibility with different platforms, primarily Docker Swarm and Kubernetes. Frameworks such as BBMA [70], Nezha [71], DESK [76], Kaiju [79], AAD [81], CHAOSORCA [82], ADMBA [85], eBPFOAS [86], and eBPFTT [88] support deployment on Docker Swarm. Docker Swarm is a container orchestration tool that allows for the management and scaling of containerized applications across a cluster of Docker hosts. Many frameworks, including MDF [65], Metrofunnel [66], MOA [67], BBMA [70], Nezha [71], KUNERVA [72], CDOF [73], ADOB [74], DESK [76], Kaiju [79], AAD [81], CHAOSORCA [82], ADMBA [85], eBPFOAS [86], and ADAM [90], support deployment on Kubernetes. Kubernetes is an open-source container orchestration platform for automating the deployment, scaling, and management of containerized applications. Some frameworks have mixed or partial support for deployment platforms. For example, MDF [65], Metrofunnel [66], and MOA [67] support Kubernetes but not Docker Swarm. On the

other hand, BBMA [70], Nezha [71], and KUNERVA [72] support both Docker Swarm and Kubernetes. Additionally, some frameworks like OESS [77] support Docker Swarm but not Kubernetes, while STEAM [80] supports Kubernetes but not Docker Swarm. Limited or Unspecified Support: Frameworks such as AET [68], SuanMing [69], MSM [78], ENMA [87], netobserv [89], and eBPFTT [88] do not specify support for either Docker Swarm or Kubernetes, indicating limited or unspecified compatibility with these deployment platforms. Ultimately, the choice of deployment platform for a specific framework depends on factors such as compatibility, scalability requirements, and organizational preferences. While Kubernetes is widely supported across many frameworks, Docker Swarm remains a viable option for those preferring simpler container orchestration solutions. Each framework's compatibility with deployment platforms contributes to its versatility and suitability for different edge computing environments and use cases. In the realm of monitoring tools utilized by various proposed frameworks for edge computing, each framework exhibits unique preferences and configurations regarding the tools integrated into its monitoring infrastructure. Frameworks like MDF [65], MOA [67], BBMA [70], Nezha [71], KUNERVA [72], Kaiju [79], CHAOSORCA [82], and ADMBA [85] incorporate Prometheus for monitoring. Prometheus is an open-source monitoring and alerting toolkit widely used for collecting and querying time-series data. Only BBMA [70] incorporates Datadog among the listed frameworks. Datadog is a cloud-based monitoring and analytics platform that integrates with various cloud providers and technologies. MDF [65], MOA [67], BBMA [70], KUNERVA [72], DESK [76], Kaiju [79], AAD [81], CHAOSORCA [82], and ADMBA [85] utilize Grafana for visualization and analytics of monitoring data. Grafana is an open-source platform for creating, exploring, and sharing dashboards and visualizations of time-series data. Metrofunnel [66], MOA [67], and Kaiju [79] utilize Zipkin for distributed tracing. Zipkin is a distributed tracing system used for monitoring and troubleshooting microservices-based architectures. BBMA [70] incorporates WeaveScope for monitoring. WeaveScope is a visualization and monitoring tool for Docker and Kubernetes clusters, providing insights into containerized applications and infrastructure. DESK [76] utilizes Loki for log aggregation and analysis. Loki is a horizontally scalable, multi-tenant log aggregation system inspired by Prometheus. STEAM [80] incorporates OpenTelemetry for observability instrumentation. OpenTelemetry is a set of APIs, libraries, agents, and instrumentation to provide observability data for cloud-native software. ADAM [90] utilizes Istio for monitoring and managing microservices. Istio is an open-source service mesh platform that provides traffic management, policy enforcement, and telemetry collection capabilities. ADAM [90] and netobserv [89] integrate Jaeger for distributed tracing. Jaeger is a distributed tracing system used for monitoring and troubleshooting microservices-based architectures. BPFTrace is a high-level tracing language for Linux extended Berkeley Packet Filter (eBPF) that allows for dynamic tracing of kernel and user-space applications. The choice of monitoring tools integrated into each framework depends on factors such as compatibility, functionality, and specific monitoring requirements. These tools collectively contribute to the observability and performance monitoring capabilities of the respective frameworks, aiding in troubleshooting, analysis, and optimization of edge computing environments.

VI. EVALUATION OF MICROSERVICES OBSERVABILITY SOLUTIONS: TOOLS, FEATURES, AND PERFORMANCE METRICS

This section provides a comprehensive evaluation of observability frameworks for microservices architecture, based on the research and findings reported in the original research papers discussed earlier. We analyze various features of these frameworks to gauge their effectiveness and efficiency in monitoring microservices within containerized environments, as outlined in table 4. Additionally, a comparative analysis of state-of-the-art observability frameworks highlights a diverse array of tools and technologies, each tailored to specific aspects such as performance monitoring, root cause analysis, anomaly detection, and alerting. The analysis reveals that Root Cause Analysis (34.2%) and Performance Analysis (30.1%) are the primary focus areas of observability frameworks, as depicted in the distribution of observability purposes across frameworks fig 4. Moreover, the feature comparison across reviewed frameworks fig 5 demonstrates varying emphases on critical aspects like Metrics, Logs, Tracing, Performance Impact, and Scalability, providing insight into the distinct design priorities of different frameworks. Lastly, the frameworks vs resource utilization fig 6 highlights disparities in CPU usage and latency among frameworks, underscoring the importance of efficiency when selecting tools for monitoring microservices.

The MDF [65] method focused on performance analysis, root cause analysis, anomaly detection, and application health by leveraging Prometheus and Grafana for monitoring and visualization. It evaluated a comprehensive set of data, including logs, metrics, and traces, although it did not address resource utilization directly. It used Stochastic Petri Net (SPN) models to analyze the data. In contrast, the Metrofunnel [66] method targeted performance analysis, root cause analysis, and application health through Zipkin and Sysdig. It primarily evaluated trace data, without focusing on resource utilization. Metrofunnel [66] employed passive tracing to unobtrusively capture system activity and container monitoring to assess the performance and health of containerized applications, offering a view into the operation of container environments. The MOA [67] method was designed for performance analysis, root cause analysis, and application health by utilizing OpenTelemetry for collecting metrics, logs, and tracing data. It specifically included CPU usage metrics and benefited from the integration of open-source monitoring frameworks, which allowed for a comprehensive

TABLE 4. Feature-based comparison of microservices observability frameworks and their performance metrics.

Framework Name	Observability Purpose	Open-source Tools	Data Evaluated	Resource Utilization	PSS	Technology Used
MDF [65]	PA, RCA, AD, AH	Prometheus, Grafana	Logs, metrics, and traces	NA	Yes	Stochastic Petri Net (SPN) models
Metrofunnel [66]	PA, RCA, AH	Zipkin, Sysdig	Traces	NA	Yes	Passive tracing, container monitoring
MOA [67]	PA, RCA, AH	OpenTelemetry	Metrics, logs, tracing	CPU usage	Yes	Open-source monitoring frameworks
AET [68]	PA, RCA, AD, AH	NA	Traces	NA	NA	Tree Abstraction
SuanMing [69]	PA, RCA	Jaeger, Zipkin	Traces	NA	Yes	Machine learning Approach
BBMA [70]	PA, RCA, AH	Prometheus, Grafana, Datalog, WeaveScope	Metrics, logs, tracing	NA	Yes	eBPF
Nezha [71]	RCA	OpenTelemetry, Prometheus, Grafana, Loki	Logs, Traces	NA	Yes	OpenTelemetry SDK & toolkit
KUNERVA [72]	PA, RCA, AD	Prometheus, Grafana	Logs	4.53% CPU, 2.58% baseline	Yes	Go language, Hubble, MySQL, MongoDB
CDOF [73]	PA, RCA, AD	NA	Metrics, logs, tracing	2.69%	Yes	eBPF
ADOB [74]	PA, RCA, AH	NA	Metrics, logs, tracing	Improves	Yes	Game theory
eBPFM [75]	PA, RCA, AD	NA	Logs	1.4% for 10,000 packets/second	Yes	eBPF
DESK [76]	PA, RCA, AD	Prometheus, Grafana, Loki, OpenTelemetry	Metrics, logs, tracing	90ms for 50 pods, 130ms for 110 pods	Yes	Apache Kafka
OESS [77]	PA, RCA, AH	NA	Metrics	Stable CPU utilization	Yes	Spring Boot, Docker engine
MSM [78]	PA, RCA, AD	NA	Logs and Traces	NA	NA	Statistical methods
Kaiju [79]	PA, RCA, AD, AH	OpenTelemetry, Jaeger, Zipkin	Metrics, logs, tracing	Increase	Yes	EPL language
STEAM [80]	PA, RCA, AH	OpenTelemetry	Traces	NA	Yes	Graph Neural Networks
AAD [81]	PA, RCA, AD	Yes	Metrics, logs, tracing	NA	Yes	Python & Docker for Kubernetes cluster
CHAOSORCA [82]	PA, RCA, AD	Jaeger, Zipkin	Metrics, logs, tracing	Almost zero for syscall	Yes	System call perturbation
eBPFNM [83]	PA	BPFtrace, Prometheus, Grafana	Metrics	CPU usage	Yes	BPFtrace
DyCause [84]	PA, RCA, AH	NA	Metrics, logs, tracing	NA	Yes	Statistical algorithm
ADMBA [85]	RCA	Prometheus, Grafana	Metrics	NA	Yes	Kubernetes platform
eBPFOAS [86]	PA, RCA	NA	Metrics, logs, tracing	Less than 1%	Yes	eBPF
ENMA [87]	PA, AH	NA	Metrics	60% with SmartNIC	Yes	eBPF
eBPFTT [88]	PA, AD	BPFTrace	Traces	NA	NA	eBPF
netobserv [89]	PA, RCA	BPFTrace	Yes	11.19%	Yes	eBPF
ADAM [90]	AD	Prometheus, Istio, Jaeger	Metrics, logs, tracing	NA	NA	Stress-ng and Kiali tools

Abbreviations and Symbols: MDF: Model Driven Framework; PSS: Proposed System Scalability; MOA: Modern Observability Approach; AET: Approach for Explainability and Traceability; BBMA: Black-box monitoring approach; CDOF: Cross-domain observability framework; ADOB: Adaptive Observability (eBPF); eBPFM: eBPF-based matcher; OESS: Observability for enhanced service sharing; MSM: Microservice Security Metrics; AAD: Automatic Anomalies Detection; ADMBA: Architecture for deploying microservice-based applications; OAS: eBPF-based observability analysis system; ENMA: Efficient Network Monitoring Applications; eBPFTT: eBPF tracing tool; netobserv: network observability with eBPF; ADAM: Anomaly Detection Approach in Microservices: eBPFNM: eBPF based Network Monitoring; PA: Performance Analysis; NA: Not Available; RCA: Root Cause Analysis; PA: performance analysis; AD: Anomaly detection **Framework Names:** Metrofunnel, SuanMing, Nezha, KUNERVA, DESK, Kaiju, STEAM, CHAOSORCA, DyCause

view of system performance. On the other hand, the AET [68] method addressed performance analysis, root cause analysis, anomaly detection, and application health through the analysis of trace data. It did not focus on resource utilization and relied on a Tree Abstraction approach, which involved organizing and representing the data in a tree-like structure to simplify the analysis of complex system behaviors and interactions. The SuanMing [69] method focused on performance analysis and root cause analysis using Jaeger and Zipkin for tracing. It did not address resource utilization directly and employed a machine-learning approach to analyze the collected traces. This technique used algorithms to detect patterns and anomalies in trace data, potentially improving the accuracy of identifying and resolving issues within the system. The BBMA [70] method covered performance analysis, root cause analysis, and application health by integrating multiple tools: Prometheus and Grafana for metrics and logs, Datalog for advanced data queries, and WeaveScope for visualizing and monitoring containerized applications. It utilized eBPF (extended Berkeley Packet Filter) technology, which provided a powerful mechanism for tracing and monitoring at a low level within the kernel. eBPF allowed for high-performance, fine-grained observability by executing small programs in response to various kernel events. This technology enhanced the ability to capture detailed performance data and system behavior, offering deep insights into application performance and resource utilization. The Nezha [71] method was designed for root cause analysis (RCA) and utilized a suite of tools including OpenTelemetry, Prometheus, Grafana, and Loki. It focused on logs and traces but did not directly address resource utilization. Nezha [71] leveraged the OpenTelemetry SDK and toolkit, which provided a comprehensive framework for collecting and exporting telemetry data. This toolkit supported a wide range of observability data, enabling detailed analysis and effective troubleshooting of issues based on logs and traces. The KUNERVA [72] and CDoF [73] methods addressed performance analysis, root cause analysis, and anomaly detection. They employed Prometheus and Grafana to monitor and visualize logs. KUNERVA [72] was notable for its focus on resource utilization, with specific metrics indicating 4.53% CPU usage and a 2.58% baseline for performance evaluation. The system was built using the Go programming language and integrated with Hubble for network visibility, MySQL for database management, and MongoDB for NoSQL data storage. This combination of technologies allowed KUNERVA [72] to efficiently perform and analyze logs, offering insights into system performance and detecting anomalies based on comprehensive data collection and analysis.

The CDoF [73] method is aimed at performance analysis (PA), root cause analysis (RCA), and anomaly detection (AD). It used metrics, logs, and tracing to gather data, with a reported resource utilization of 2.69%. CDoF employed eBPF (extended Berkeley Packet Filter) technology, which provided high-performance, low-overhead observability by

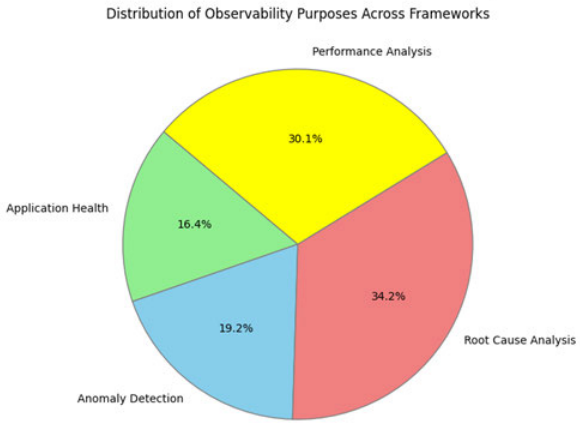

FIGURE 4. Distribution of monitoring purpose of the frameworks.

allowing programs to be run in the kernel space. This approach enabled detailed monitoring and tracing of system activities at a granular level, enhancing the ability to detect and analyze anomalies efficiently. The ADOB [74] method focused on performance analysis, root cause analysis, and application health (AH) using metrics, logs, and tracing data. It was noted for its ability to improve performance insights and issue resolution. ADOB [74] utilized game theory, which involved applying mathematical models of strategic interactions to optimize decision-making performances. In the context of observability, game theory was used to model and predict system behavior, allocate resources effectively, and make informed decisions based on the strategic interactions between different system components. This approach helped in understanding complex system dynamics and improving overall system health and performance. The eBPFM [75] method targeted performance analysis (PA), root cause analysis (RCA), and anomaly detection (AD). It utilized logs and was noted for its resource utilization of 1.4% when performing 10,000 packets per second. eBPFM [75] employed eBPF (extended Berkeley Packet Filter) technology, which allowed for high-performance, low-overhead monitoring by executing small programs in the kernel. This enabled detailed and efficient analysis of network traffic and system behavior, enhancing the detection and investigation of anomalies.

The DESK [76] method addressed performance analysis, root cause analysis, and anomaly detection using a suite of tools including Prometheus, Grafana, Loki, and OpenTelemetry. It evaluated metrics, logs, and tracing data, and was capable of monitoring performance with specific timing metrics: 90 milliseconds for 50 pods and 130 milliseconds for 110 pods. These timings reflected the system's responsiveness and efficiency in performing observability data across different scales of deployment. The integration of these tools allowed DESK [76] to provide comprehensive observability by aggregating data from various sources, enabling effective analysis and quick detection of issues in containerized environments. The OESS [77] method focused on performance analysis (PA),

root cause analysis (RCA), and application health (AH) using metrics. It was designed to maintain stable CPU utilization, indicating that it effectively managed system resources while monitoring performance. OESS [77] operated with Spring Boot for application development and Docker Engine for containerization, allowing for effective deployment and monitoring of applications in a containerized environment. The MSM [78] method was aimed at performance analysis (PA), root cause analysis (RCA), and anomaly detection (AD). It relied on logs and traces for data collection and did not directly specify resource utilization. MSM [78] employed statistical methods to analyze the collected data. These methods involved applying statistical techniques to identify patterns, trends, and anomalies in the logs and traces. By using statistical models and algorithms, MSM [78] could detect deviations from normal behavior, quantify the likelihood of certain issues, and provide insights into the system's performance and potential problems. This approach enhanced the ability to predict and diagnose issues based on historical data and statistical analysis. The Kaiju [79] method targeted performance analysis (PA), root cause analysis (RCA), anomaly detection (AD), and application health (AH). It utilized OpenTelemetry, Jaeger, and Zipkin for collecting metrics, logs, and tracing data. Kaiju [79] was designed to enhance observability by leveraging these tools to gather comprehensive performance data. It focused on increasing visibility into system operations and employed the EPL (Event Performing Language) for querying and analyzing the collected data. EPL enabled complex event performing by allowing users to define and execute queries on event streams, providing advanced capabilities for detecting and responding to events in real time.

The STEAM [80] method addressed performance analysis (PA), root cause analysis (RCA), and application health (AH) using OpenTelemetry for tracing data collection. It did not directly focus on resource utilization and relied on Graph Neural Networks (GNNs) for analysis. GNNs were a type of machine learning model designed to work with graph-structured data. They were particularly effective in capturing complex relationships and dependencies within the data. In the context of STEAM [80], GNNs analyzed the tracing data to identify patterns and anomalies, model dependencies between different system components, and provide insights into the system's behavior. This approach enhanced the ability to detect issues and understand system dynamics by leveraging the power of graph-based models to analyze and interpret observability data. The AAD [81] method focused on performance analysis (PA), root cause analysis (RCA), and anomaly detection (AD). It collected a variety of observability data, including metrics, logs, and tracing information, though it did not specify its impact on resource utilization. The method utilized Python and Docker for its operations within Kubernetes clusters, leveraging these tools to manage and analyze data efficiently. Python scripts were often used to perform and interpret the collected data, while Docker ensured that the environment was consistent and scalable across different deployments. This combination facilitated effective monitoring and observability.

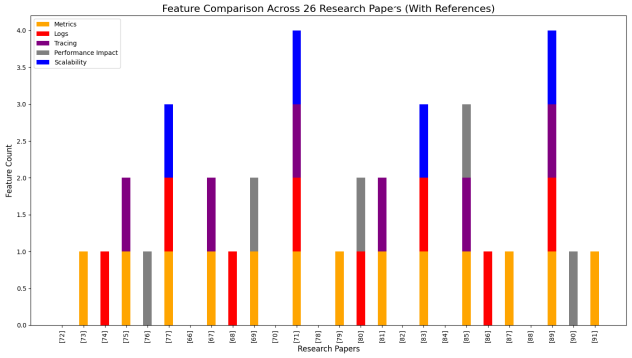

FIGURE 5. Frameworks' feature comparison.

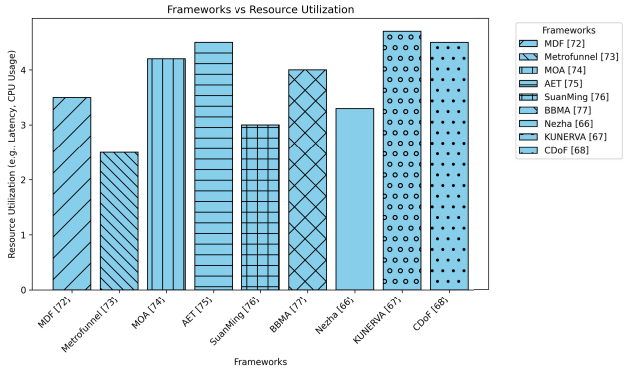

FIGURE 6. Resource utilization among the frameworks.

The scatter plot fig 7 provides a comparative analysis of the observability tools employed in research studies focused on microservices. The most frequently utilized tool is eBPF, mentioned in research papers discussed in section V, highlighting its growing popularity for efficient, kernel-level monitoring with minimal overhead. This reflects a trend towards leveraging low-level system capabilities for deeper insights into performance and diagnostics. These tools have become essential components in monitoring frameworks, offering comprehensive visibility into specifically microservice environments.

Tracing solutions are also well-represented, with OpenTelemetry appearing in 5 studies, and Jaeger and Zipkin each referenced in 4 papers. This suggests a strong emphasis on distributed tracing to capture the flow of requests across microservices, which is critical for identifying performance bottlenecks and diagnosing complex issues. The increasing use of OpenTelemetry indicates a shift towards standardization and integration of metrics, logs, and traces within a single observability framework.

Specialized tools like BPFTrace, mentioned in 3 papers, showcase the demand for dynamic tracing techniques that offer detailed system insights. Tools such as Loki, Sysdig, WeaveScope, Apache Kafka, and Istio are less frequently cited, suggesting their use in specific scenarios or as supplementary components in broader observability

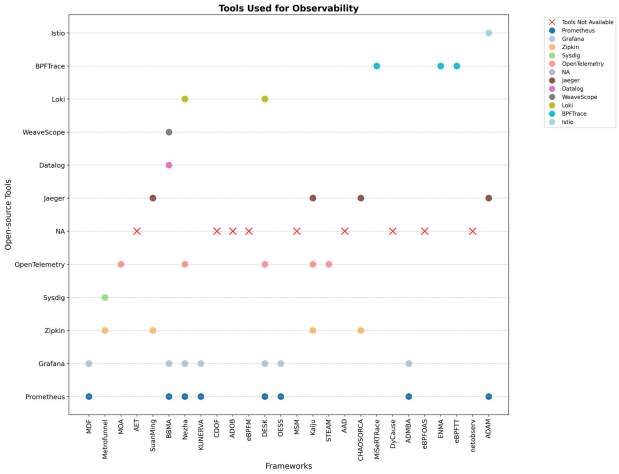

FIGURE 7. Usage frequency of the observability tools.

solutions. Overall, the data points to a strong preference for open-source, community-supported tools like Prometheus, Grafana, and OpenTelemetry, which provide flexibility and integration capabilities vital for complex microservices architectures. The emphasis on kernel-level observability and distributed tracing further underscores the industry's focus on achieving comprehensive monitoring and diagnostic capabilities.

A. ADVANCES AND LIMITATIONS IN EXISTING MICROSERVICES OBSERVABILITY FRAMEWORKS

In this subsection, we evaluated and synthesized key insights from our comparative analysis of observability frameworks for microservices. Many state-of-the-art solutions, such as Prometheus, Grafana, and Zipkin, offer comprehensive monitoring capabilities, integrating logs, metrics, and traces for efficient performance analysis and anomaly detection. Advanced frameworks like STEAM [80] and ADOB [74] employ cutting-edge techniques, such as game theory and Graph Neural Networks (GNNs), to enhance system behavior analysis. Additionally, technologies like eBPF enable low-overhead, high-performance observability in frameworks like BBMA [70], making them well-suited for resource-constrained environments.

However, significant gaps remain in several critical areas of observability for microservices, highlighting the need for further advancements. One major gap is in resource utilization monitoring. While some frameworks, such as KUNERVA [72] and netobserv [89], make strides in addressing resource management, these solutions often fall short in providing a comprehensive view of resource consumption across diverse distributed environments. Effective resource utilization monitoring is crucial for optimizing performance and controlling costs, especially in large-scale deployments where efficient use of computational resources directly impacts system scalability and operational efficiency. There is a pressing need for more robust solutions that can offer granular visibility into resource usage, provide actionable insights for optimization, and integrate seamlessly with various distributed system architectures.

Another critical challenge is real-time observability at scale. Although machine learning-driven anomaly detection techniques, like those employed in the SuanMing framework, offer promising capabilities for identifying complex patterns and anomalies, there is still significant room for improvement. These methods need to be optimized to handle the dynamic nature of large-scale systems effectively. As microservices and distributed environments continue to grow in size and complexity, real-time monitoring and analysis become increasingly challenging. Future research should focus on developing scalable, high-performance solutions that can maintain real-time observability and adapt to the rapidly changing conditions in modern distributed systems.

Additionally, interoperability across diverse platforms remains a significant hurdle. As applications and services are deployed across various environments—such as cloud infrastructures, edge computing nodes, and IoT devices—observability frameworks must be capable of seamlessly integrating and operating across these heterogeneous systems. Current solutions often struggle with this interoperability, leading to fragmented visibility and difficulties in managing observability across different platforms. Enhanced frameworks that can provide a unified view and support seamless integration across diverse environments will be crucial for effective monitoring and management of modern distributed systems. Further open research issues and challenges will be discussed in the next section, where we will explore these unresolved aspects.

VII. OPEN RESEARCH ISSUES AND FUTURE DIRECTION

This section delves into the open research issues and challenges related to observability in microservices, drawing on insights from our comparative study. As microservices architectures become increasingly prevalent, addressing these challenges is crucial for advancing the field of observability and ensuring that systems remain efficient, reliable, and scalable. Here are some key areas that require further exploration and
development:

A. COMPREHENSIVE OBSERVABILITY ACROSS LEVELS

Achieving thorough observability across all levels—system, services, and network—presents a significant challenge. Most existing frameworks tend to focus primarily on one or two of these levels, often overlooking the network level, which is vital for diagnosing issues in distributed systems. Integrating observability tools that offer insights at all these levels is essential. This integration must be done in a way that avoids disrupting system performance and is efficient, especially timely in dynamic microservices environments where configurations and states can frequently change [97].

B. UNIFIED DATA COLLECTION AND ANALYSIS

A key aspect of effective observability is the ability to integrate various types of data—logs, metrics, and traces—to create a comprehensive understanding of the system's condition. This holistic view is crucial for diagnosing issues and detecting anomalies in microservices architectures [98]. While some methods have made progress in this area, there remains a need for improved tools and techniques that can seamlessly correlate data from different sources and types in real-time. This integration is necessary for accurate root cause analysis and timely detection of issues without imposing significant performance overhead.

C. SCALABILITY AND REAL-TIME PROCESSING

As microservices architectures become more extensive and complex, observability tools must be able to handle increasing amounts of data efficiently [99]. Scalability is a major concern, as these tools must grow alongside the systems they monitor without degrading performance. Moreover, real-time processing capabilities are essential to provide immediate feedback and enable quick responses to detected issues. Ensuring that observability tools can scale effectively while maintaining the ability to process data in real-time is crucial for managing large-scale environments.

D. ADAPTABILITY TO DYNAMIC ENVIRONMENTS

Microservices environments are highly dynamic, with frequent updates in service deployments and configurations. Observability frameworks need to be flexible enough to adapt to these changes to maintain accurate monitoring. This means developing systems that can automatically detect new services, adjust monitoring parameters on the fly, and continue to provide precise insights despite ongoing changes. Creating observability systems that can seamlessly adapt to these evolving conditions is vital for effective monitoring in rapidly changing environments.

E. PRIVACY AND SECURITY

In this section, we have discussed the research challenges related to privacy and security in microservices observability across different levels.

1) KERNEL LEVEL

Granularity vs. Privacy: At the kernel level, accessing detailed system information and resource usage metrics may provide valuable observability insights. However, this level of granularity raises concerns about data privacy, as it may expose sensitive information about system processes and interactions. Balancing the need for detailed observability with preserving user privacy poses a significant research challenge [100].

Secure Kernel Instrumentation: Implementing observability tools that interact directly with the kernel introduces security risks, as any vulnerabilities in these tools could potentially be exploited to gain unauthorized access to sensitive system resources. Research is needed to develop robust security mechanisms for kernel-level observability, including secure instrumentation techniques and access controls [101], [102].

2) USER LEVEL

Personal Data Protection: Observability at the user level involves monitoring user interactions with microservices, which may include sensitive personal data. Ensuring the privacy and security of this data is paramount, as unauthorized access or exposure could result in privacy violations and legal ramifications. Research is needed to develop techniques for anonymizing or pseudonymizing user data while still enabling effective observability [102], [103].

Identity and Access Management: User-level observability requires robust identity and access management (IAM) practices to control access to sensitive user data. Research challenges include designing secure authentication mechanisms, enforcing the principle of least privilege, and preventing unauthorized data access by malicious actors or insider threats. [78].

3) NETWORK LEVEL

Data in Transit Protection: Observability at the network level involves monitoring communication between microservices, which may include the transmission of sensitive data over the network. Research is needed to develop secure communication protocols, such as encryption and authentication mechanisms, to protect data in transit from interception or tampering. Security has always been an issue in networking systems, but it gets more difficult with the use of microservices [104].

Network Traffic Analysis: Analyzing network traffic patterns and anomalies for observability purposes raises privacy concerns, as it may inadvertently reveal sensitive information about system architecture, data flows, or user behavior. Research challenges include developing techniques for network traffic analysis that balance the need for observability with preserving user privacy, such as aggregation and anonymization methods.

To address these challenges, future work should focus on specific areas for further exploration. Addressing the comprehensive observability across all system, service, and network levels will require developing unified tools that offer deeper insights without impacting performance. Refining existing frameworks to incorporate more network-level observability could help overcome gaps in diagnosing distributed systems. In terms of unified data collection and analysis, there's a need for tools that can seamlessly correlate logs, metrics, and traces in real time to enable accurate root cause analysis without imposing significant overhead. Additionally, as microservices architectures grow in complexity, tools must be enhanced to handle increasing data volumes, improving scalability and real-time processing capabilities. Observability frameworks should also

evolve to become more adaptable to dynamic microservices environments, incorporating automation and AI to detect and adjust monitoring parameters in response to frequent changes. On the privacy and security front [58], more secure methods for kernel-level instrumentation and stronger identity and real-time management at the user level are required. For network-level observability, future research must focus on developing encryption and traffic analysis techniques that prioritize security and privacy. Addressing these areas will provide the foundation for more effective and secure observability in microservices architectures.

F. POTENTIAL OPPORTUNITIES FOR EMERGING RESEARCHERS

Future research in microservices observability can explore self-adaptive observability frameworks that dynamically adjust monitoring strategies based on workload patterns, system anomalies, and resource constraints. Unlike static observability configurations, such adaptive frameworks would leverage reinforcement learning and feedback loops to optimize metric collection, trace sampling, and log aggregation in real-time, reducing overhead while preserving critical insights. Additionally, integrating causal inference techniques into observability pipelines could enhance root cause analysis by distinguishing correlation from causation in complex microservices failures. These directions align with the evolving need for intelligent, automated, and efficient observability solutions that can scale with the increasing complexity of distributed cloud-native architectures.

VIII. CONCLUSION

This paper presents a comprehensive analysis of the transition towards microservices architecture in containerized cloud, fog, and edge environments, emphasizing the critical need for enhanced observability to sustain system reliability and performance. We have examined various sophisticated observability mechanisms that are essential for the prompt diagnosis and mitigation of potential failures. These mechanisms are indispensable to ensure continuous service availability and maintain optimal performance levels, which are crucial for user satisfaction and operational continuity. Our analysis reveals a significant gap in current observability frameworks, particularly in their capacity to manage the increasing complexity and dynamic nature of modern distributed systems. In addition, the study suggests that future developments should prioritize the scalability and adaptability of observability tools to keep pace with the rapid evolution of microservices architectures. As systems grow and change, these tools must evolve correspondingly to provide consistent and reliable insights across various deployment environments, from the cloud to the fog and edge. In essence, advancing observability frameworks is pivotal not only for maintaining the reliability and performance of microservices architectures but also for ensuring that they can effectively meet the evolving demands of users and technological landscapes. This will ultimately contribute to the resilience and efficiency of modern digital infrastructures that support a wide array of applications and services. Therefore, future research should focus on developing comprehensive, scalable, and adaptable observability tools that can meet the challenges posed by the continuous evolution of microservices in computing environments.

ACKNOWLEDGMENT

The authors extend their thanks to the National University of Computer and Emerging Sciences (FAST), Karachi, Pakistan, for their valuable technical assistance. Lastly, they are deeply appreciative of the anonymous reviewers for their detailed feedback and insightful suggestions, which have substantially improved the overall quality and presentation of this article.

CONFLICT OF INTEREST

The authors declare that there are no conflicts of interest regarding the publication of this article. No financial, personal, or other relationships with organizations or individuals have influenced the content or findings presented in this manuscript.

REFERENCES

[1] S. Luo, C. Lin, K. Ye, G. Xu, L. Zhang, G. Yang, H. Xu, and C. Xu, "Optimizing resource management for shared microservices: A scalable system design," *ACM Trans. Comput. Syst.*, vol. 42, nos. 1–2, pp. 1–28, May 2024.

[2] S. Long, Y. Zhang, Q. Deng, T. Pei, J. Ouyang, and Z. Xia, "An efficient task offloading approach based on multi-objective evolutionary algorithm in cloud-edge collaborative environment," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 2, pp. 645–657, Mar. 2023.

[3] H. J. Syed, A. Gani, R. W. Ahmad, M. K. Khan, and A. I. A. Ahmed, "Cloud monitoring: A review, taxonomy, and open research issues," *J. Netw. Comput. Appl.*, vol. 98, pp. 11–26, Nov. 2017.

[4] I. Shabani, E. Mëziu, B. Berisha, and T. Biba, "Design of modern distributed systems based on microservices architecture," *Int. J. Adv. Comput. Sci. Appl.*, vol. 12, no. 2, pp. 1–7, 2021.

[5] I. K. Aksakalli, T. Çelik, A. B. Can, and B. Tekinerdoğan, "Deployment and communication patterns in microservice architectures: A systematic literature review," *J. Syst. Softw.*, vol. 180, Oct. 2021, Art. no. 111014.

[6] F. Aydemir and F. Başçiftçi, "Building a performance efficient core banking system based on the microservices architecture," *J. Grid Comput.*, vol. 20, no. 4, p. 37, Dec. 2022.

[7] J. S. Nunes, S. C. Sampaio, R. S. Malaquias, and I. De Morais Barroca Filho, "Deploying the observability of the SigSaude system using service mesh," in *Proc. 20th Int. Conf. Comput. Sci. Its Appl. (ICCSA)*, Jul. 2020, pp. 9–15.

[8] A. Martínez Saucedo, G. Rodríguez, F. Gomes Rocha, and R. P. D. Santos, "Migration of monolithic systems to microservices: A systematic mapping study," *Inf. Softw. Technol.*, vol. 177, Jan. 2025, Art. no. 107590.

[9] G. Liu, B. Huang, Z. Liang, M. Qin, H. Zhou, and Z. Li, "Microservices: Architecture, container, and challenges," in *Proc. IEEE 20th Int. Conf. Softw. Quality, Rel. Secur. Companion (QRS-C)*, Dec. 2020, pp. 629–635.

[10] C. Cassé, P. Berthou, P. Owezarski, and S. Josset, "A tracing based model to identify bottlenecks in physically distributed applications," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Jan. 2022, pp. 226–231.

[11] T. Zeng, Z. Xu, D. Wu, X. Li, B. Liu, H. Hu, S. Tan, Y. Tan, C. Xu, C. Stewart, Q. Zhou, and Y. Cao, "Exploring nonintrusive measurements of spatio-temporal portrait of microservices," *Softw., Pract. Exp.*, vol. 54, no. 10, pp. 2025–2041, Oct. 2024.

[12] Y. Wang, H. Kadiyala, and J. Rubin, "Promises and challenges of microservices: An exploratory study," *Empirical Softw. Eng.*, vol. 26, no. 4, p. 63, Jul. 2021.

[13] H. Siddiqui, F. Khendek, and M. Toeroe, "Microservices based architectures for IoT systems—State-of-the-art review," *Internet Things*, vol. 23, Oct. 2023, Art. no. 100854.

[14] A. B. Raharjo, P. K. Andyartha, W. H. Wijaya, Y. Purwananto, D. Purwitasari, and N. Juniarta, "Reliability evaluation of microservices and monolithic architectures," in *Proc. Int. Conf. Comput. Eng., Netw., Intell. Multimedia (CENIM)*, Nov. 2022, pp. 1–7.

[15] R. Gatev and R. Gatev, "Observability: Logs, metrics, and traces," in *Introducing Distributed Application Runtime (Dapr): Simplifying Microservices Applications Development Through Proven and Reusable Patterns and Practices*. Berkeley, CA, USA: Apress, 2021, pp. 233–252.

[16] B. Li, X. Peng, Q. Xiang, H. Wang, T. Xie, J. Sun, and X. Liu, "Enjoy your observability: An industrial survey of microservice tracing and analysis," *Empirical Softw. Eng.*, vol. 27, no. 1, pp. 1–28, Jan. 2022.

[17] D. Das, M. Schiewe, E. Brighton, M. Fuller, T. Cerny, M. Bures, K. Frajtak, D. Shin, and P. Tisnovsky, "Failure prediction by utilizing log analysis: A systematic mapping study," in *Proc. Int. Conf. Res. Adapt. Convergent Syst.*, Oct. 2020, pp. 188–195.

[18] L. Wang, N. Zhao, J. Chen, P. Li, W. Zhang, and K. Sui, "Root-cause metric location for microservice systems via log anomaly detection," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, Oct. 2020, pp. 142–150.

[19] E. A. Matusse, E. H. M. Huzita, and T. F. C. Tait, "Metrics and indicators to assist in the distribution of process steps for distributed software development: A systematic review," in *Proc. Conferencia Latinoamericana En Inf. (CLEI)*, Oct. 2012, pp. 1–10.

[20] M. G. Moreira and B. B. N. De França, "Analysis of microservice evolution using cohesion metrics," in *Proc. 16th Brazilian Symp. Softw. Compon., Architectures, Reuse*, Oct. 2022, pp. 40–49.

[21] E. Fadda, P. Plebani, and M. Vitali, "Monitoring-aware optimal deployment for applications based on microservices," *IEEE Trans. Services Comput.*, vol. 14, no. 6, pp. 1849–1863, Nov. 2021.

[22] M. Söylemez, B. Tekinerdogan, and A. Kolukısa Tarhan, "Challenges and solution directions of microservice architectures: A systematic literature review," *Appl. Sci.*, vol. 12, no. 11, p. 5507, May 2022.

[23] J. Rios, S. Jha, and L. Shwartz, "Localizing and explaining faults in microservices using distributed tracing," in *Proc. IEEE 15th Int. Conf. Cloud Comput. (CLOUD)*, Jul. 2022, pp. 489–499.

[24] S. Alam, S. Zardari, S. Noor, S. Ahmed, and H. Mouratidis, "Trust management in social Internet of Things (SIoT): A survey," *IEEE Access*, vol. 10, pp. 108924–108954, 2022.

[25] S. Zhang, M. Zhang, L. Ni, and P. Liu, "A multi-level self-adaptation approach for microservice systems," in *Proc. IEEE 4th Int. Conf. Cloud Comput. Big Data Anal. (ICCCBDA)*, Apr. 2019, pp. 498–502.

[26] J. Bogner, S. Schlinger, S. Wagner, and A. Zimmermann, "A modular approach to calculate service-based maintainability metrics from runtime data of microservices," in *Proc. Int. Conf. Product-Focused Softw. Process Improvement*. Cham, Switzerland: Springer, Jan. 2019, pp. 489–496.

[27] J. Kosinska, B. Balis, M. Konieczny, M. Malawski, and S. Zielinski, "Towards the observability of cloud-native applications: The overview of the state-of-the-art," *IEEE Access*, vol. 11, pp. 73036–73052, 2023.

[28] M. Stocker, O. Zimmermann, U. Zdun, D. Lübke, and C. Pautasso, "Interface quality patterns: Communicating and improving the quality of microservices Apis," in *Proc. 23rd Eur. Conf. Pattern Lang. Programs*, Jul. 2018, pp. 1–16.

[29] D. Berardi, S. Giallorenzo, J. Mauro, A. Melis, F. Montesi, and M. Prandini, "Microservice security: A systematic literature review," *PeerJ Comput. Sci.*, vol. 7, p. e779, Jan. 2022.

[30] A. Bhardwaj and T. A. Benson, "KubeKlone: A digital twin for simulating edge and cloud microservices," in *Proc. 6th Asia–Pacific Workshop Netw.*, Jul. 2022, pp. 29–35.

[31] J. Soldani and A. Brogi, "Anomaly detection and failure root cause analysis in (Micro) service-based cloud applications: A survey," *ACM Comput. Surveys*, vol. 55, no. 3, pp. 1–39, Mar. 2023.

[32] M. Asim, Y. Wang, K. Wang, and P.-Q. Huang, "A review on computational intelligence techniques in cloud and edge computing," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 4, no. 6, pp. 742–763, Dec. 2020.

[33] Y. Mansouri and M. A. Babar, "A review of edge computing: Features and resource virtualization," *J. Parallel Distrib. Comput.*, vol. 150, pp. 155–183, Apr. 2021.

[34] C. Mwase, Y. Jin, T. Westerlund, H. Tenhunen, and Z. Zou, "Communication-efficient distributed AI strategies for the IoT edge," *Future Gener. Comput. Syst.*, vol. 131, pp. 292–308, Jun. 2022.

[35] X. Liu, Y. Yu, M. Yu, D. Liao, Y. Li, and Z. Ji, "An edge-cloud collaborative computing system for real-time Internet-of-Things applications," in *Proc. Int. Conf. Comput. Sci. Educ.* Cham, Switzerland: Springer, 2022, pp. 652–663.

[36] V. Veeramachaneni, "Edge computing: Architecture, applications, and future challenges in a decentralized era," *Recent Trends Comput. Graph. Multimedia Technol.*, vol. 7, no. 1, pp. 8–23, 2025.

[37] G. Carvalho, B. Cabral, V. Pereira, and J. Bernardino, "Edge computing: Current trends, research challenges and future directions," *Computing*, vol. 103, no. 5, pp. 993–1023, May 2021.

[38] A. Dimou, C. Iliopoulos, E. Polytidou, S. K. Dhurandher, G. Papadimitriou, and P. Nicopolitidis, "A comprehensive review on edge computing: Focusing on mobile users," in *Advances in Computing, Informatics, Networking and Cybersecurity: A Book Honoring Professor Mohammad S. Obaidat's Significant Scientific Contributions*, 2022, pp. 121–152.

[39] J. Almutairi and M. Aldossary, "A novel approach for IoT tasks offloading in edge-cloud environments," *J. Cloud Comput.*, vol. 10, no. 1, p. 28, Apr. 2021.

[40] T. Zhang, Y. Li, and C. L. Philip Chen, "Edge computing and its role in industrial Internet: Methodologies, applications, and future directions," *Inf. Sci.*, vol. 557, pp. 34–65, May 2021.

[41] F. Firouzi, B. Farahani, and A. Marinšek, "The convergence and interplay of edge, fog, and cloud in the AI-driven Internet of Things (IoT)," *Inf. Syst.*, vol. 107, Jul. 2022, Art. no. 101840.

[42] H. G. Abreha, C. J. Bernardos, A. D. L. Oliva, L. Cominardi, and A. Azcorra, "Monitoring in fog computing: State-of-the-art and research challenges," *Int. J. Ad Hoc Ubiquitous Comput.*, vol. 36, no. 2, pp. 114–130, 2021.

[43] B. Costa, J. Bachiega, L. R. Carvalho, M. Rosa, and A. Araujo, "Monitoring fog computing: A review, taxonomy and open challenges," *Comput. Netw.*, vol. 215, Oct. 2022, Art. no. 109189.

[44] D. Soldani, P. Nahi, H. Bour, S. Jafarizadeh, M. F. Soliman, L. Di Giovanna, F. Monaco, G. Ognibene, and F. Risso, "EBPF: A new approach to cloud-native observability, networking and security for current (5G) and future mobile networks (6G and beyond)," *IEEE Access*, vol. 11, pp. 57174–57202, 2023.

[45] T. Weng, W. Yang, G. Yu, P. Chen, J. Cui, and C. Zhang, "Kmon: An in-kernel transparent monitoring system for microservice systems with eBPF," in *Proc. IEEE/ACM Int. Workshop Cloud Intell. (CloudIntelligence)*, May 2021, pp. 25–30.

[46] B. Sharma and D. Nadig, "EBPF-enhanced complete observability solution for cloud-native microservices," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Jun. 2024, pp. 1980–1985.

[47] M. R. S. Sedghpour and P. Townend, "Service mesh and eBPF-powered microservices: A survey and future directions," in *Proc. IEEE Int. Conf. Service-Oriented Syst. Eng. (SOSE)*, Aug. 2022, pp. 176–184.

[48] Y.-T. Wang, S.-P. Ma, Y.-J. Lai, and Y.-C. Liang, "Analyzing and monitoring Kubernetes microservices based on distributed tracing and service mesh," in *Proc. 29th Asia–Pacific Softw. Eng. Conf. (APSEC)*, Dec. 2022, pp. 477–481.

[49] S. Miano, F. Risso, M. V. Bernal, M. Bertrone, and Y. Lu, "A framework for eBPF-based network functions in an era of microservices," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 1, pp. 133–151, Mar. 2021.

[50] E. Muškardin, M. Tappler, B. K. Aichernig, and I. Pill, "Reinforcement learning under partial observability guided by learned environment models," in *Proc. Int. Conf. Integr. Formal Methods*. Cham, Switzerland: Springer, Jan. 2022, pp. 257–276.

[51] L. Min, K. A. Alnowibet, A. F. Alrasheedi, F. Moazzen, E. M. Awwad, and M. A. Mohamed, "A stochastic machine learning based approach for observability enhancement of automated smart grids," *Sustain. Cities Soc.*, vol. 72, Sep. 2021, Art. no. 103071.

[52] X. Wu, Z. Jing, and X. Wang, "The security of IoT from the perspective of the observability of complex networks," *Heliyon*, vol. 10, no. 5, Mar. 2024, Art. no. e27104.

[53] C. Ding, L. Zhu, L. Shen, Z. Li, Y. Li, and Q. Liang, "The intelligent traffic flow control system based on 6G and optimized genetic algorithm," *IEEE Trans. Intell. Transp. Syst.*, early access, Oct. 7, 2024, doi: 10.1109/TITS.2024.3467269.

[54] X. Cong, M. P. Fanti, A. M. Mangini, and Z. Li, "Critical observability verification and enforcement of labeled Petri nets by using basis markings," *IEEE Trans. Autom. Control*, vol. 68, no. 12, pp. 8158–8164, Dec. 2023.

[55] M. Akbari, R. Bolla, R. Bruschi, F. Davoli, C. Lombardo, and B. Siccardi, "A monitoring, observability and analytics framework to improve the sustainability of B5G technologies," in *Proc. IEEE Int. Conf. Commun. Workshops (ICC Workshops)*, Jun. 2024, pp. 969–975.

[56] M. Usman, S. Ferlin, A. Brunstrom, and J. Taheri, "A survey on observability of distributed edge & container-based microservices," *IEEE Access*, vol. 10, pp. 86904–86919, 2022.

[57] J. Parmar, S. Sanghavi, V. K. Prasad, and P. Shah, "Microservice architecture observability tool analysis," in *Proc. Int. Conf. Soft Comput. Signal Process.* Cham, Switzerland: Springer, Jan. 2023, pp. 1–8.

[58] S. Li, H. Zhang, Z. Jia, C. Zhong, C. Zhang, Z. Shan, J. Shen, and M. A. Babar, "Understanding and addressing quality attributes of microservices architecture: A systematic literature review," *Inf. Softw. Technol.*, vol. 131, Mar. 2021, Art. no. 106449.

[59] V. Velepucha and P. Flores, "A survey on microservices architecture: Principles, patterns and migration challenges," *IEEE Access*, vol. 11, pp. 88339–88358, 2023.

[60] S. Zhang, S. Xia, W. Fan, B. Shi, X. Xiong, Z. Zhong, M. Ma, Y. Sun, and D. Pei, "Failure diagnosis in microservice systems: A comprehensive survey and analysis," *ACM Trans. Softw. Eng. Methodology*, Jan. 2025.

[61] M. Hamza, "Transforming monolithic systems to a microservices architecture," *ACM SIGSOFT Softw. Eng. Notes*, vol. 48, no. 1, pp. 67–69, Jan. 2023.

[62] A. Razzaq and S. A. K. Ghayyur, "A systematic mapping study: The new age of software architecture from monolithic to microservice architecture—Awareness and challenges," *Comput. Appl. Eng. Educ.*, vol. 31, no. 2, pp. 421–451, Mar. 2023.

[63] R. Su, X. Li, and D. Taibi, "From microservice to monolith: A multivocal literature review," *Electronics*, vol. 13, no. 8, p. 1452, Apr. 2024.

[64] L. Giamattei, A. Guerriero, R. Pietrantuono, S. Russo, I. Malavolta, T. Islam, M. Dînga, A. Koziolek, S. Singh, M. Armbruster, J. M. Gutierrez-Martinez, S. Caro-Alvaro, D. Rodriguez, S. Weber, J. Henss, E. F. Vogelin, and F. S. Panojo, "Monitoring tools for DevOps and microservices: A systematic grey literature review," *J. Syst. Softw.*, vol. 208, Feb. 2024, Art. no. 111906.

[65] I. Fé, T. A. Nguyen, A. B. Soares, S. Son, E. Choi, D. Min, J.-W. Lee, and F. A. Silva, "Model-driven dependability and power consumption quantification of Kubernetes-based cloud-fog continuum," *IEEE Access*, vol. 11, pp. 140826–140852, 2023.

[66] M. Cinque, R. D. Corte, and A. Pecchia, "Microservices monitoring with event logs and black box execution tracing," *IEEE Trans. Services Comput.*, vol. 15, no. 1, pp. 294–307, Jan. 2022.

[67] I. Tzanettis, C.-M. Androna, A. Zafeiropoulos, E. Fotopoulou, and S. Papavassiliou, "Data fusion of observability signals for assisting orchestration of distributed applications," *Sensors*, vol. 22, no. 5, p. 2061, Mar. 2022.

[68] P.-J. Quéval and U. Zdun, "Extracting the architecture of microservices: An approach for explainability and traceability," in *Proc. Eur. Conf. Softw. Archit.* Cham, Switzerland: Springer, Jan. 2023, pp. 346–353.

[69] J. Grohmann, M. Straesser, A. Chalbani, S. Eismann, Y. Arian, N. Herbst, N. Peretz, and S. Kounev, "SuanMing: Explainable prediction of performance degradations in microservice applications," in *Proc. ACM/SPEC Int. Conf. Perform. Eng.*, Apr. 2021, pp. 165–176.

[70] R. Brondolin and M. D. Santambrogio, "A black-box monitoring approach to measure microservices runtime performance," *ACM Trans. Archit. Code Optim.*, vol. 17, no. 4, pp. 1–26, Dec. 2020.

[71] G. Yu, P. Chen, Y. Li, H. Chen, X. Li, and Z. Zheng, "Nezha: Interpretable fine-grained root causes analysis for microservices on multi-modal observability data," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2023, pp. 553–565.

[72] S. Lee and J. Nam, "Kunerva: Automated network policy discovery framework for containers," *IEEE Access*, vol. 11, pp. 95616–95631, 2023.

[73] K. Ranjitha, P. Tammana, P. G. Kannan, and P. Naik, "A case for cross-domain observability to debug performance issues in microservices," in *Proc. IEEE 15th Int. Conf. Cloud Comput. (CLOUD)*, Jul. 2022, pp. 244–246.

[74] D. Monteiro, Y. Yu, A. Zisman, and B. Nuseibeh, "Adaptive observability for forensic-ready microservice systems," *IEEE Trans. Services Comput.*, vol. 16, no. 5, pp. 3196–3209, Sep./Oct. 2023.

[75] L. Wüstrich, M. Schacherbauer, M. Budeus, D. Freiherr von Künßberg, S. Gallenmüller, M.-O. Pahl, and G. Carle, "Network profiles for detecting application-characteristic behavior using Linux eBPF," in *Proc. 1st Workshop eBPF Kernel Extensions*, Sep. 2023, pp. 8–14.

[76] M. Usman, S. Ferlin, A. Brunstrom, and J. Taheri, "Desk: Distributed observability framework for edge-based containerized microservices," in *Proc. Joint Eur. Conf. Netw. Commun. 6G Summit (EuCNC/6G Summit)*, Jun. 2023, pp. 617–622.

[77] K. Alanezi and S. Mishra, "Utilizing microservices architecture for enhanced service sharing in IoT edge environments," *IEEE Access*, vol. 10, pp. 90034–90044, 2022.

[78] U. Zdun, P.-J. Queval, G. Simhandl, R. Scandariato, S. Chakravarty, M. Jelic, and A. Jovanovic, "Microservice security metrics for secure communication, identity management, and observability," *ACM Trans. Softw. Eng. Methodology*, vol. 32, no. 1, pp. 1–34, Jan. 2023.

[79] M. Scrocca, R. Tommasini, A. Margara, E. D. Valle, and S. Sakr, "The kaiju project: Enabling event-driven observability," in *Proc. 14th ACM Int. Conf. Distrib. Event-based Syst.*, Jul. 2020, pp. 85–96.

[80] S. He, B. Feng, L. Li, X. Zhang, Y. Kang, Q. Lin, S. Rajmohan, and D. Zhang, "STEAM: Observability-preserving trace sampling," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2023, pp. 1750–1761.

[81] O. Mart, C. Negru, F. Pop, and A. Castiglione, "Observability in Kubernetes cluster: Automatic anomalies detection using prometheus," in *Proc. IEEE 22nd Int. Conf. High Perform. Comput. Commun., IEEE 18th Int. Conf. Smart City, IEEE 6th Int. Conf. Data Sci. Syst. (HPCC/SmartCity/DSS)*, Dec. 2020, pp. 565–570.

[82] J. Simonsson, L. Zhang, B. Morin, B. Baudry, and M. Monperrus, "Observability and chaos engineering on system calls for containerized applications in Docker," *Future Gener. Comput. Syst.*, vol. 122, pp. 117–129, Sep. 2021.

[83] L.-D. Chou, L.-Y. Jian, and Y.-W. Chen, "EBPF-based network monitoring platform on Kubernetes," in *Proc. 6th Int. Conf. Comput. Commun. Internet (ICCCI)*, Jun. 2024, pp. 140–144.

[84] Y. Pan, M. Ma, X. Jiang, and P. Wang, "Faster, deeper, easier: Crowdsourcing diagnosis of microservice kernel failure from user space," in *Proc. 30th ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2021, pp. 646–657.

[85] L. Abdollahi Vayghan, M. A. Saied, M. Toeroe, and F. Khendek, "Deploying microservice based applications with Kubernetes: Experiments and lessons learned," in *Proc. IEEE 11th Int. Conf. Cloud Comput. (CLOUD)*, Jul. 2018, pp. 970–973.

[86] C. Liu, Z. Cai, B. Wang, Z. Tang, and J. Liu, "A protocol-independent container network observability analysis system based on eBPF," in *Proc. IEEE 26th Int. Conf. Parallel Distrib. Syst. (ICPADS)*, Dec. 2020, pp. 697–702.

[87] M. Abranches, O. Michel, E. Keller, and S. Schmid, "Efficient network monitoring applications in the kernel with eBPF and XDP," in *Proc. IEEE Conf. Netw. Function Virtualization Softw. Defined Netw. (NFV-SDN)*, Nov. 2021, pp. 28–34.

[88] C. Cassagnes, L. Trestioreanu, C. Joly, and R. State, "The rise of eBPF for non-intrusive performance monitoring," in *Proc. NOMS-IEEE/IFIP Netw. Oper. Manage. Symp.*, Apr. 2020, pp. 1–7.

[89] P. G. Kannan, S. M. Gupta, D. Behl, E. Raichstein, and J. Takvorian, "Designing a lightweight network observability agent for cloud applications," in *Proc. Int. Conf. Passive Act. Netw. Meas.* Cham, Switzerland: Springer, Jan. 2024, pp. 262–276.

[90] M. Panahandeh, A. Hamou-Lhadj, M. Hamdaqa, and J. Miller, "ServiceAnomaly: An anomaly detection approach in microservices using distributed traces and profiling metrics," *J. Syst. Softw.*, vol. 209, Mar. 2024, Art. no. 111917.

[91] T. Shiraishi, M. Noro, R. Kondo, Y. Takano, and N. Oguchi, "Real-time monitoring system for container networks in the era of microservices," in *Proc. 21st Asia–Pacific Netw. Oper. Manage. Symp. (APNOMS)*, Sep. 2020, pp. 161–166.

[92] J. Levin and T. A. Benson, "ViperProbe: Rethinking microservice observability with eBPF," in *Proc. IEEE 9th Int. Conf. Cloud Netw. (CloudNet)*, Nov. 2020, pp. 1–8.

[93] R. Rangaiyengar, R. Komondoor, and R. K. Medicherla, "Multi-layer observability for fault localization in microservices based systems," in *Proc. IEEE Int. Conf. Softw. Anal., Evol. Reengineering (SANER)*, Mar. 2023, pp. 733–737.

[94] V. Raj and G. P. Chander, "Monitoring of microservices architecture based applications using process mining," in *Proc. 9th Int. Conf. Comput. Sustain. Global Develop. (INDIACom)*, Mar. 2022, pp. 486–494.
[95] C. H. Zhang and M. Omair Shafiq, "A real-time, scalable monitoring and user analytics solution for microservices-based software applications," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Dec. 2022, pp. 6125–6134.
[96] N. D. Keni and A. Kak, "Adaptive containerization for microservices in distributed cloud systems," in *Proc. IEEE 17th Annu. Consum. Commun. Netw. Conf. (CCNC)*, Jan. 2020, pp. 1–6.
[97] D. Ernst and S. Tai, "Offline trace generation for microservice observability," in *Proc. IEEE 25th Int. Enterprise Distrib. Object Comput. Workshop (EDOCW)*, Oct. 2021, pp. 308–317.
[98] D. Calavera and L. Fontana, *Linux Observability With BPF: Advanced Programming for Performance Analysis and Networking*. Sebastopol, CA, USA: O'Reilly Media, 2019.
[99] S. Baškarada, V. Nguyen, and A. Koronios, "Architecting microservices: Practical opportunities and challenges," *J. Comput. Inf. Syst.*, vol. 60, no. 5, pp. 428–436, Sep. 2020.
[100] D. Gorige, E. Al-Masri, S. Kanzhelev, and H. Fattah, "Privacy-risk detection in microservices composition using distributed tracing," in *Proc. IEEE Eurasia Conf. IoT, Commun. Eng. (ECICE)*, Oct. 2020, pp. 250–253.
[101] N. Mateus-Coelho, M. Cruz-Cunha, and L. G. Ferreira, "Security in microservices architectures," *Proc. Comput. Sci.*, vol. 181, pp. 1225–1236, Jul. 2021.
[102] Y. He, R. Guo, Y. Xing, X. Che, K. Sun, Z. Liu, K. Xu, and Q. Li, "Cross container attacks: The bewildered eBPF on clouds," in *Proc. 32nd USENIX Secur. Symp. (USENIX Secur.)*, 2023, pp. 5971–5988.
[103] M. Driss, D. Hasan, W. Boulila, and J. Ahmad, "Microservices in IoT security: Current solutions, research challenges, and future directions," *Proc. Comput. Sci.*, vol. 192, pp. 2385–2395, Jul. 2021.
[104] A. Hannousse and S. Yahiouche, "Securing microservices and microservice architectures: A systematic mapping study," *Comput. Sci. Rev.*, vol. 41, Aug. 2021, Art. no. 100415.

UMMAY FASEEHA (Member, IEEE) is currently pursuing the Ph.D. degree with the Department of Computer Science, National University of Computer and Emerging Sciences (FAST), Karachi, Pakistan. She is an Assistant Professor with Jinnah University for Women, Karachi. With a strong commitment to academic excellence, she has been involved in various research projects and has made significant contributions to the field of computer science through her teaching and research endeavors. Her research interests include machine learning, embedded systems, SDN, NFV, eBPF, and the Internet of Things.

HASSAN JAMIL SYED received the bachelor's degree in electrical engineering from the Quaid-e-Awam University of Engineering, Sciences, and Technology, Nawabshah, Pakistan, in 2003, the master's degree in personal mobile and satellite communication from the University of Bradford, U.K., in 2008, and the Ph.D. degree in computer science from the University of Malaya, Malaysia, in 2019. After completing his Ph.D., he was an Assistant Professor with the Department of Computer Sciences, National University of Computer and Emerging Sciences, Karachi, Pakistan, from August 2019 to June 2022. He was a Senior Lecturer with the Faculty of Computing and Informatics, Universiti Malaysia Sabah (UMS), Sabah, Malaysia, for two years. He also worked with WorldCall Telecom Ltd., Karachi, as a Network Engineer for two years, and with the Faculty of Engineering Science and Technology, Iqra University, Karachi, for four years. He is currently a Senior Lecturer with the School of Technology, Asia Pacific University of Innovation and Technology (APU), Kuala Lumpur, Malaysia. Throughout his career, he has supervised several Ph.D./M.S. thesis/final year projects and taught courses in the electronics, telecommunication, and computer science departments. His research interests include cloud computing, cybersecurity, SDN, NFV, eBPF, and the Internet of Things.

FAHAD SAMAD received the Ph.D. degree from RWTH Aachen University (specialization: network security). He is eagerly looking forward to academic and industrial opportunities and challenges ahead. His objective is to be actively involved and to always play a major role in shaping the technically advanced society using the skill set that he has developed over the past several years.

SEHAR ZEHRA received the Master of Science degree in computer science from the Sir Syed University of Engineering and Technology, in 2012. She is currently pursuing the Ph.D. degree in computer science with the National University of Computer and Emerging Sciences (FAST), Karachi, Pakistan. She has amassed 20 years of teaching experience and is a full-time Assistant Professor with the Khursheed Government Girls Degree College, Karachi. Over her extensive teaching career, she has contributed significantly to her students' academic and professional growth, supervising numerous projects, and teaching various courses across the curriculum.

HAMZA AHMED is currently pursuing the Ph.D. degree in computer science with the National University of Computer and Emerging Sciences (FAST), Karachi, Pakistan. He has a rich background in academia, having served as a Lecturer with FAST University for four years before transitioning to teach A-level students with Karachi Grammar School for two years. Alongside his Ph.D. studies, he is actively involved in teaching as a Visiting Faculty Member at various universities. His extensive experience in education has allowed him to contribute significantly to the academic and professional development of his students.